

Laporan Akhir Proyek Big Data

Pengembangan Sistem Big Data untuk Analisis Earnings dan Sales
Estimate Saham Menggunakan Apache Spark, SQL, dan NoSQL

Kelompok 02



Oleh:

11423024	:	Fernando Yosia Parhusip
11423026	:	Flora Nainggolan
11423031	:	Febri Sweeta Br Lumban Raja
11423041	:	Dewa Armada Gurusinga
11423051	:	Erwin Jeremy Sianturi

D4 TEKNOLOGI REKAYASA PERANGKAT LUNAK
FAKULTAS VOKASI
INSTITUT TEKNOLOGI DEL

DAFTAR ISI

1. Latar Belakang.....	6
2. Pembagian Task Per Orang.....	8
3. Dataset.....	8
4. Arsitektur Sistem.....	9
5. Implementasi.....	10
5.1 Data Ingestion.....	11
5.2 Exploratory Data Analyst.....	11
5.2.1 Informasi Dasar Dataset.....	12
5.2.2 Pemeriksaan Missing Value.....	13
5.2.3 Pemeriksaan Data Duplikat.....	14
5.2.4 Statistik Deskriptif Kolom Numerik.....	14
5.2.5 Pemeriksaan Nilai Tidak Logis.....	15
5.2.6 Distribusi Periode Estimasi.....	16
5.2.7 Top 10 Saham Berdasarkan Jumlah Data.....	16
5.2.8 Histogram & Boxplot.....	17
5.2 Data Cleaning dan Preprocessing.....	19
5.3 Data Processing Menggunakan Apache Spark.....	21
5.3.1 Pembuatan Kolom Analisis.....	21
5.3.2 Ringkasan Berdasarkan Periode Estimasi.....	22
5.3.3 Analisis Saham Berdasarkan Estimasi Rata-Rata.....	23
5.3.4 Analisis Saham Berdasarkan Estimasi Paling Stabil.....	24
5.3.5 Analisis Saham Berdasarkan Pertumbuhan Estimasi.....	25
5.3.6 Output Data Processing.....	26
5.4 Integrasi Database SQL.....	28

5.4.1 Konfigurasi Koneksi Database.....	28
5.4.2 Pembuatan Tabel Secara Otomatis.....	29
5.4.3 Penyimpanan Data ke Database.....	30
5.4.4 Penyimpanan Data ke Database.....	30
5.4.5 Hasil Integrasi dan Penyimpanan Data.....	31
5.5 Integrasi NoSQL.....	32
5.5.1 Koneksi Database SQL dan MongoDB.....	32
5.5.2 Pengambilan Data dari Database SQL.....	33
5.5.3 Penyimpanan Data ke MongoDB.....	34
5.5.4 Migrasi Data dari SQL ke NoSQL.....	34
5.5.4 Migrasi Data dari SQL ke NoSQL.....	35
6. Hasil dan Analisis.....	36
6.1 Analisis Distribusi Data Berdasarkan Periode Estimasi.....	36
6.2 Analisis Saham dengan Estimasi Rata-rata tertinggi.....	36
6.3 Analisis Saham dengan Estimasi Paling Stabil.....	37
6.4 Analisis Top 10 Saham dengan Pertumbuhan Estimasi Tertinggi.....	38
6.5 Analisis 10 Saham Estimasi Tertinggi pada Periode Kuartal Berjalan.....	38
6.6 Analisis 10 Saham Estimasi Tertinggi pada Periode Kuartal Berikutnya.....	39
6.7 Analisis 10 Saham Estimasi Tertinggi pada Periode Tahun Berjalan.....	40
6.8 Analisis 10 Saham Estimasi Tertinggi pada Periode Tahun Berikutnya.....	40
6.9 Saham dengan Analisis Terbanyak Kuartal Berjalan.....	41
6.10 Analisis 10 Saham dengan Analisis Terbanyak Kuartal Berikutnya.....	42
6.11 Analisis 10 Saham dengan Analisis Terbanyak Tahun Berjalan.....	43
6.12 Analisis 10 Saham dengan Analisis Terbanyak Tahun Berikutnya.....	44
6.13 Pembahasan Dashboard Streamlit.....	45

7. Kesimpulan.....	47
--------------------	----

DAFTAR GAMBAR

Gambar 1 Arsitektur Sistem	10
Gambar 2 Pembuatan SparkSession	11
Gambar 3 Proses download dataset	12
Gambar 4 Informasi Dasar Dataset	13
Gambar 5 Pemeriksaan Missing Value	14
Gambar 6 Pemeriksaan Data Duplikat	15
Gambar 7 Statistik Deskriptif Kolom Numerik	15
Gambar 8 Distribusi Periode Estimasi	17
Gambar 9 Top 10 Saham Berdasarkan Dataset	17
Gambar 10 Estimasi Rata-Rata	18
Gambar 11 Boxplot Kolom Numerik	19
Gambar 12 Hasil Data Cleaning & Preprocessing	21
Gambar 13 Pembuatan Kolom Analisis	22
Gambar 14 Ringkasan Berdasarkan Periode Estimasi	23
Gambar 15 Ringkasan Berdasarkan Periode Estimasi	24
Gambar 16 Analisis Saham Berdasarkan Estimasi Paling Stabil	25
Gambar 17 Analisis Saham Berdasarkan Pertumbuhan Estimasi	26
Gambar 18 Output Hasil Processing	27

Gambar 19 Konfigurasi Koneksi Database	28
Gambar 20 Pembuatan Tabel Secara Otomatis	29
Gambar 21 Penyimpanan Data ke Database	30
Gambar 22 Penyimpanan Data ke Database	30
Gambar 23 Hasil Integrasi dan Penyimpanan Data	31
Gambar 24 Koneksi Database SQL dan MongoDB	32
Gambar 25 Pengambilan Data dari Database SQL	33
Gambar 26 Penyimpanan Data ke MongoDB	34
Gambar 27 Migrasi Data dari SQL ke NoSQL	34
Gambar 28 Migrasi Data dari SQL ke NoSQL	35
Gambar 29 Analisis Distribusi Data Berdasarkan Periode Estimasi	36
Gambar 30 Analisis Saham dengan Estimasi Rata-rata tertinggi	37
Gambar 31 Analisis Saham dengan Estimasi Paling Stabil	37
Gambar 32 Analisis Top 10 Saham dengan Pertumbuhan Estimasi Tertinggi	38
Gambar 33 Saham Estimasi Tertinggi pada Periode Kuartal Berjalan	39
Gambar 34 Analisis 10 Saham Estimasi Tertinggi pada Periode Kuartal Berikutnya	39
Gambar 35 Analisis 10 Saham Estimasi Tertinggi pada Periode Tahun Berjalan	40
Gambar 36 Saham Estimasi Tertinggi pada Periode Tahun Berikutnya	41
Gambar 37 Saham dengan Analisis Terbanyak Kuartal Berjalan	42
Gambar 38 Analisis 10 Saham dengan Analisis Terbanyak Kuartal Berikutnya	43
Gambar 39 Analisis 10 Saham dengan Analisis Terbanyak Tahun Berjalan	44
Gambar 40 Saham dengan Analisis Terbanyak Tahun Berikutnya	45
Gambar 41 Pembahasan Dashboard Streamlit	46

DAFTAR TABEL

Tabel 1 Pembagian Task Per Orang	9
Tabel 2 Deskripsi Dataset	10

1. Latar Belakang

Perkembangan teknologi informasi membuat jumlah data yang dihasilkan dalam berbagai bidang semakin besar, termasuk pada bidang keuangan dan pasar saham. Data saham tidak hanya berisi harga transaksi, tetapi juga mencakup data estimasi seperti earnings estimate dan sales estimate. Data tersebut dapat digunakan untuk melihat gambaran estimasi pendapatan dan penjualan suatu perusahaan dalam periode tertentu. Namun, karena jumlah data yang sangat besar, proses pengolahan dan analisis tidak efektif jika dilakukan secara manual atau menggunakan cara konvensional.

Pada proyek ini, dataset yang digunakan memiliki jumlah data awal sebanyak **6.618.616** record. Setelah dilakukan proses cleaning dan preprocessing, jumlah data menjadi **5.714.890** record, sehingga terdapat **903.726** record yang dihapus karena tidak sesuai dengan kebutuhan pengolahan data. Jumlah data yang besar ini menunjukkan bahwa proyek membutuhkan teknologi yang mampu memproses data dalam skala besar secara efisien.

Untuk mengatasi permasalahan tersebut, proyek ini membangun sistem Big Data yang memanfaatkan Apache Spark sebagai teknologi utama dalam proses pengolahan data. Apache Spark digunakan untuk melakukan data ingestion, data cleaning, preprocessing, dan processing terhadap dataset. Hasil pengolahan data kemudian disimpan ke dalam database relasional menggunakan MySQL/TiDB dan diintegrasikan ke database non-relasional menggunakan MongoDB. Integrasi antara SQL dan NoSQL dilakukan agar sistem dapat menunjukkan alur penyimpanan dan pengelolaan data yang lebih fleksibel.

Selain melakukan pengolahan dan penyimpanan data, sistem ini juga menghasilkan analisis berupa ringkasan data berdasarkan periode estimasi, saham dengan estimasi rata-rata tertinggi, saham dengan pertumbuhan tertinggi, serta saham yang memiliki tingkat kestabilan lebih baik berdasarkan rentang estimasi. Hasil analisis tersebut kemudian divisualisasikan menggunakan dashboard agar informasi yang dihasilkan lebih mudah dipahami.

Dengan adanya sistem ini, data mentah dalam jumlah besar dapat diubah menjadi informasi yang lebih ringkas, terstruktur, dan informatif. Sistem yang dibangun tidak bertujuan untuk memberikan rekomendasi investasi secara langsung, tetapi digunakan sebagai alat bantu untuk melakukan eksplorasi dan analisis awal terhadap data earnings estimate dan sales estimate saham. Proyek ini juga sesuai dengan kebutuhan mata kuliah Big Data yang menekankan proses ingestion, processing, integrasi SQL dan NoSQL, analisis, serta visualisasi data.

2. Pembagian Task Per Orang

Tabel 1.1 merupakan tabel pembagian tugas masing masing anggota tim.

Tabel 1 Pembagian Task Per Orang

Nama	Fokus Tugas	Detail Pekerjaan	Bagian Presentasi
Erwin Jeremy Sianturi	Perencanaan Sistem, Latar Belakang, dan Dataset	Menjelaskan latar belakang, tujuan proyek, alasan memilih data earnings dan sales estimate saham, sumber dataset, struktur dataset, dan jumlah data awal.	Latar belakang, tujuan proyek, dataset, dan data awal 6.618.616 record.
Flora Nainggolan	Data Ingestion dan Apache Spark Setup	Menjelaskan proses mengambil dataset, membaca data CSV ke Google Colab, setup SparkSession, dan ingestion data ke Spark DataFrame.	Proses ingestion data dan alasan menggunakan Apache Spark.
Fernando Yosia Parhusip	Data Cleaning dan Processing	Menjelaskan preprocessing, penghapusan data kosong/duplikat/tidak valid, transformasi data, dan hasil data cleaning.	Data awal, data setelah cleaning 5.714.890 record, dan data terhapus 903.726 record.

Nama	Fokus Tugas	Detail Pekerjaan	Bagian Presentasi
Febri Sweeta Br Lumban Raja	Integrasi SQL dan NoSQL	Menjelaskan penyimpanan hasil Spark ke MySQL/TiDB, transfer data dari MySQL ke MongoDB, dan verifikasi collection MongoDB.	Arsitektur sistem dan alur integrasi Spark MySQL/TiDB MongoDB.
Dewa Armada Gurusinga	Query Analysis, Visualisasi, dan Dashboard	Menjelaskan query analysis dari MongoDB, dashboard Streamlit, grafik, tabel hasil query, insight, dan kesimpulan.	Hasil analisis, dashboard Streamlit, insight, dan kesimpulan proyek.

3. Dataset

Dataset yang digunakan pada proyek ini merupakan dataset estimasi pendapatan dan penjualan saham. Dataset tersebut berisi data dalam jumlah besar yang digunakan untuk menganalisis estimasi saham berdasarkan beberapa atribut, seperti kode saham, periode estimasi, estimasi rata-rata, rentang estimasi, dan pertumbuhan dibanding tahun sebelumnya.

Dataset diproses menggunakan Apache Spark karena jumlah data yang digunakan sangat besar. Jumlah data awal pada proyek ini adalah 6.618.616 record. Setelah dilakukan proses cleaning dan preprocessing, jumlah data menjadi 5.714.890 record. Dengan demikian, sebanyak 903.726 record dihapus karena mengandung nilai kosong, duplikat, atau data yang tidak sesuai kebutuhan analisis.

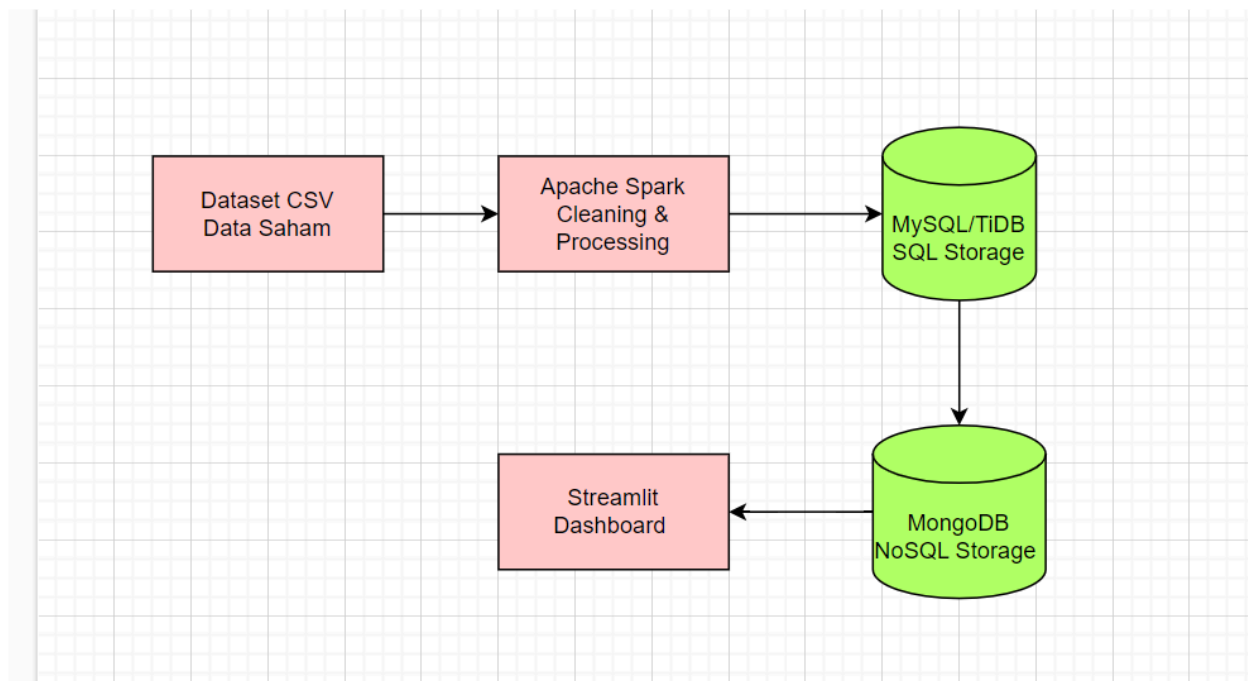
Tabel 2 Deskripsi Dataset

Nama Kolom	Tipe Data	Deskripsi
Tanggal	DateTime	Tanggal data dicatat atau dipublikasikan
kode_saham	String	Kode saham perusahaan, misalnya AMZN, AAPL
periode_estimasi	String	Periode estimasi pendapatan atau laporan keuangan, seperti Current Quarter, Current Year, Next Quarter, dan Next Year
tanggal_akhir_period	DateTime	Tanggal akhir periode laporan keuangan yang diperkirakan.
estimasi_rata_rata	Numerik (<i>Float</i>)	Nilai rata-rata estimasi dari seluruh analis terhadap pendapatan/performa perusahaan.

Nama Kolom	Tipe Data	Deskripsi
jumlah_analis	Integer	Jumlah analis yang memberikan estimasi terhadap saham tersebut.
estimasi_tertinggi	Numerik (<i>Float</i>)	Nilai estimasi tertinggi yang diberikan analis.
estimasi_terendah	Numerik (<i>Float</i>)	Nilai estimasi terendah yang diberikan analis.
estimasi_tahun_lalu	Numerik (<i>Float</i>)	Nilai aktual atau estimasi pada periode yang sama di tahun sebelumnya sebagai pembandingan historis.

4. Arsitektur Sistem

Arsitektur sistem pada proyek ini menggunakan alur Big Data end-to-end. Dataset CSV data saham diproses menggunakan Apache Spark untuk tahap cleaning dan processing. Hasil pemrosesan disimpan ke MySQL/TiDB sebagai database SQL, kemudian dipindahkan ke MongoDB sebagai database NoSQL. Data dari MongoDB selanjutnya digunakan sebagai sumber visualisasi pada dashboard Streamlit.



Gambar 1 Arsitektur Sistem

5. Implementasi

Implementasi sistem dilakukan secara bertahap untuk membangun pipeline Big Data end-to-end. Tahapan dimulai dari data ingestion, yaitu membaca *dataset earnings* dan *sales estimate* saham ke dalam *Apache Spark*. Selanjutnya dilakukan data *cleaning* dan *preprocessing* untuk menghapus data yang tidak valid, data kosong, dan duplikasi. Setelah data bersih, dilakukan data *processing* menggunakan *Apache Spark* untuk menghasilkan beberapa data ringkasan dan hasil analisis awal.

Hasil processing kemudian disimpan ke database SQL menggunakan MySQL/TiDB. Setelah itu, data dari MySQL/TiDB dibaca kembali dan ditransfer ke MongoDB sebagai database NoSQL. Proses ini digunakan untuk menunjukkan integrasi antara database SQL dan NoSQL. Setelah data tersedia di MongoDB, dilakukan query analysis untuk memperoleh insight seperti distribusi periode estimasi, saham dengan estimasi tertinggi, saham paling stabil, dan saham dengan pertumbuhan tertinggi. Visualisasi menggunakan Streamlit difokuskan pada analisis kuartalan, yaitu Kuartal Berjalan dan Kuartal Berikutnya, sedangkan hasil analisis lainnya disajikan dalam bentuk tabel dan insight pada laporan.

5.1 Data Ingestion

Tahap awal implementasi dilakukan dengan membuat *SparkSession* sebagai titik masuk utama untuk menggunakan *Apache Spark*. *SparkSession* digunakan untuk membaca, memproses, membersihkan, dan menganalisis dataset earnings dan sales estimate saham dalam skala besar. Setelah *SparkSession* berhasil dibuat, sistem menampilkan versi Spark yang digunakan.

```
# Membuat SparkSession sebagai mesin utama untuk mengolah data besar
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("BigDataProject") \
    .getOrCreate()
print("SparkSession berhasil dibuat")
print(spark.version)

... SparkSession berhasil dibuat
4.0.2
```

Gambar 2 Pembuatan SparkSession

Kode pada Gambar 5.2 digunakan untuk mengambil dataset CSV dari *Google Drive* dan menyimpannya ke direktori *Google Collab*. Dataset yang telah berhasil diunduh kemudian digunakan sebagai input pada proses pembacaan data menggunakan *Apache Spark*.

```
!pip install gdown
import os
import gdown
import re

download_url = "https://drive.google.com/file/d/1kLb0zkgHWJAvbU_hGXNsL8yAUyL7e5ya/view?usp=sharing"
local_file_name = "estimasi_Pendapatan_Penjualan_Saham.csv"
file_path = os.path.join("/content", local_file_name)

print(f"Attempting to download file from: {download_url}")
print(f"Saving to local path: {file_path}")
```

Gambar 3 Proses download dataset

5.2 Exploratory Data Analyst

Exploratory Data Analysis (EDA) dilakukan setelah dataset berhasil dimuat ke dalam *Apache Spark DataFrame*. Tahap ini bertujuan untuk memahami kondisi awal dataset sebelum dilakukan *proses cleaning* dan preprocessing. Analisis awal dilakukan dengan memeriksa jumlah data dan kolom, struktur schema, contoh data, missing value, data duplikat, statistik deskriptif kolom numerik, distribusi periode estimasi, serta validasi nilai yang tidak logis.

5.2.1 Informasi Dasar Dataset

Dataset yang digunakan terdiri dari 6.618.616 record dan 9 kolom. Jumlah data ini telah memenuhi ketentuan proyek Big Data karena melebihi batas minimal 100.000 record. Hasil

schema menunjukkan bahwa beberapa kolom numerik seperti `estimasi_rata_rata`, `estimasi_tertinggi`, `estimasi_terendah`, dan `estimasi_tahun_lalu` telah terbaca sebagai tipe `double`, sedangkan kolom `Tanggal` dan `tanggal_akhir_periode` masih bertipe `string`.

```
# EXPLORATORY DATA ANALYSIS (EDA)
# Dataset Earnings dan Sales Estimate Saham

from pyspark.sql.functions import col, count, when, trim, lower, desc
import matplotlib.pyplot as plt

# Fungsi bantu untuk format angka Indonesia
def format_angka(nilai):
    return f"{nilai:,}".replace(",", ".")

print("EDA dimulai...")

# 1. Informasi Dasar Dataset
jumlah_data = df_raw.count()
jumlah_kolom = len(df_raw.columns)

print("Jumlah data :", format_angka(jumlah_data))
print("Jumlah kolom:", jumlah_kolom)

print("\nDaftar kolom:")
for kolom in df_raw.columns:
    print("-", kolom)

# 2. Schema Dataset
print("Schema dataset:")
df_raw.printSchema()

# 3. Contoh Data Awal
print("Contoh 5 baris pertama:")
df_raw.show(5, truncate=False)
```

```

Jumlah data : 6.618.616
Jumlah kolom: 9

Daftar kolom:
- Tanggal
- kode_saham
- periode_estimasi
- tanggal_akhir_periode
- estimasi_rata_rata
- jumlah_analisis
- estimasi_tertinggi
- estimasi_terendah
- estimasi_tahun_lalu
Schema dataset:
root
|-- Tanggal: string (nullable = true)
|-- kode_saham: string (nullable = true)
|-- periode_estimasi: string (nullable = true)
|-- tanggal_akhir_periode: string (nullable = true)
|-- estimasi_rata_rata: double (nullable = true)
|-- jumlah_analisis: double (nullable = true)
|-- estimasi_tertinggi: double (nullable = true)
|-- estimasi_terendah: double (nullable = true)
|-- estimasi_tahun_lalu: double (nullable = true)

Contoh 5 baris pertama:
+-----+-----+-----+-----+-----+-----+-----+-----+
|Tanggal|kode_saham|periode_estimasi|tanggal_akhir_periode|estimasi_rata_rata|jumlah_analisis|estimasi_tertinggi|estimasi_terendah|estimasi_tahun_lalu|
+-----+-----+-----+-----+-----+-----+-----+-----+
|10/26/17 12:00 AM|A|Current Quarter|10/31/17 12:00 AM|1.17E9|7.0|1.19E9|1.16E9|1.11E9|
|10/26/17 12:00 AM|A|Current Year|10/31/17 12:00 AM|4.45E9|8.0|4.47E9|4.44E9|4.2E9|
|10/26/17 12:00 AM|A|Next Quarter|1/31/18 12:00 AM|1.14E9|3.0|1.15E9|1.13E9|1.07E9|
|10/26/17 12:00 AM|A|Next Year|10/31/18 12:00 AM|4.72E9|8.0|4.76E9|4.68E9|4.45E9|
|10/26/17 12:00 AM|AA|Current Quarter|12/31/17 12:00 AM|3.37E9|2.0|3.44E9|3.29E9|2.54E9|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Gambar 4 Informasi Dasar Dataset

5.2.2 Pemeriksaan Missing Value

Hasil pemeriksaan missing value menunjukkan bahwa kolom identitas seperti Tanggal, kode_saham, periode_estimasi, dan tanggal_akhir_periode tidak memiliki nilai kosong. Namun, beberapa kolom numerik memiliki missing value yang cukup besar, terutama jumlah_analisis, estimasi_tertinggi, dan estimasi_terendah sebanyak 903.726 record. Oleh karena itu, tahap cleaning diperlukan sebelum data digunakan untuk analisis.

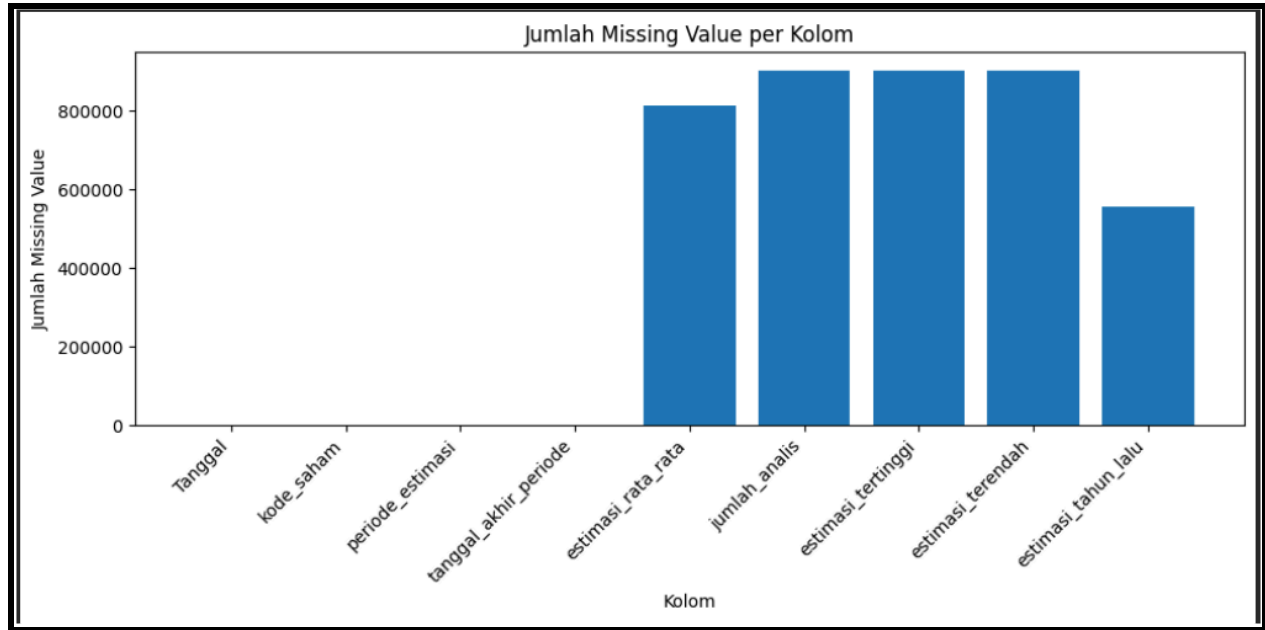
```

# 4. Mengecek Missing Value
missing_df = df_raw.select([
    count(
        when(
            col(c).isNull() |
            (trim(col(c).cast("string")) == "") |
            (lower(trim(col(c).cast("string"))) == "nan"),
            c
        )
    ).alias(c)
    for c in df_raw.columns
])

print("Jumlah missing value per kolom:")
missing_df.show()

Jumlah missing value per kolom:
+-----+-----+-----+-----+-----+-----+-----+-----+
|Tanggal|kode_saham|periode_estimasi|tanggal_akhir_periode|estimasi_rata_rata|jumlah_analisis|estimasi_tertinggi|estimasi_terendah|estimasi_tahun_lalu|
+-----+-----+-----+-----+-----+-----+-----+-----+
|      0|      0|      0|      0|      813452|      903726|      903726|      903726|      555735|
+-----+-----+-----+-----+-----+-----+-----+-----+

```



Gambar 5 Pemeriksaan Missing Value

5.2.3 Pemeriksaan Data Duplikat

Berdasarkan hasil pengecekan duplikasi, tidak ditemukan data duplikat pada dataset. Jumlah data awal sama dengan jumlah data unik, yaitu 6.618.616 record. Hal ini menunjukkan bahwa setiap baris data bersifat unik dan tidak terdapat baris yang sama persis.

```
# 6. Mengecek Data Duplikat
jumlah_data_awal = df_raw.count()
jumlah_data_unik = df_raw.dropDuplicates().count()
jumlah_duplikat = jumlah_data_awal - jumlah_data_unik

print("Jumlah data awal :", format_angka(jumlah_data_awal))
print("Jumlah data unik :", format_angka(jumlah_data_unik))
print("Jumlah duplikat :", format_angka(jumlah_duplikat))

Jumlah data awal : 6.618.616
Jumlah data unik : 6.618.616
Jumlah duplikat : 0
```

Gambar 6 Pemeriksaan Data Duplikat

5.2.4 Statistik Deskriptif Kolom Numerik

Statistik deskriptif menunjukkan bahwa nilai estimasi memiliki rentang yang sangat besar. Nilai maksimum pada estimasi_tertinggi mencapai 2.38454E12, sedangkan beberapa kolom seperti estimasi_terendah dan estimasi_tahun_lalu memiliki nilai minimum negatif. Nilai negatif tidak

langsung dianggap salah karena dalam konteks earnings, nilai negatif dapat menunjukkan estimasi kerugian. Namun, rentang nilai yang sangat besar menunjukkan adanya nilai ekstrem atau outlier.

```
# 7. Statistik Deskriptif Kolom Numerik
kolom_numerik = [
    "estimasi_rata_rata",
    "jumlah_analis",
    "estimasi_tertinggi",
    "estimasi_terendah",
    "estimasi_tahun_lalu"
]

print("Statistik deskriptif kolom numerik:")
df_raw.select(kolom_numerik).describe().show()
```

Statistik deskriptif kolom numerik:

	estimasi_rata_rata	jumlah_analis	estimasi_tertinggi	estimasi_terendah	estimasi_tahun_lalu
count	5805164	5714890	5714890	5714890	6062881
mean	4.555277010809686E9	3.4792373956454106	4.764653540099634E9	4.494830079361108E9	4.231808696319456E9
stddev	2.089526114676590...	2.723319988941225	2.172708004438837E10	2.050849126657139E10	1.939540045651768E10
min	0.0	1.0	0.0	-9.98E8	-9.9892E8
max	7.9631E11	38.0	2.38454E12	7.8184E11	7.1492E11

Gambar 7 Statistik Deskriptif Kolom Numerik

5.2.5 Pemeriksaan Nilai Tidak Logis

Hasil validasi menunjukkan bahwa tidak terdapat data dengan estimasi_tertinggi lebih kecil dari estimasi_terendah. Selain itu, tidak ditemukan nilai negatif pada kolom jumlah_analis. Hal ini menunjukkan bahwa dataset tidak memiliki kesalahan logika dasar pada dua aspek tersebut.

```
# 8. Validasi Nilai Tidak Logis

invalid_estimasi = df_raw.filter(
    col("estimasi_tertinggi") < col("estimasi_terendah")
).count()

invalid_jumlah_analis = df_raw.filter(
    col("jumlah_analis") < 0
).count()

print("Jumlah data dengan estimasi tertinggi < estimasi terendah:",
      format_angka(invalid_estimasi))

print("Jumlah data dengan jumlah analis negatif:",
      format_angka(invalid_jumlah_analis))

Jumlah data dengan estimasi tertinggi < estimasi terendah: 0
Jumlah data dengan jumlah analis negatif: 0
```

Gambar 5.7 Pemeriksaan Nilai Tidak Logis

5.2.6 Distribusi Periode Estimasi

Distribusi data berdasarkan periode_estimasi menunjukkan bahwa jumlah data pada setiap periode sama, yaitu 1.654.654 record. Hal ini menunjukkan bahwa dataset memiliki distribusi periode yang seimbang sehingga analisis berdasarkan periode dapat dilakukan dengan lebih adil.

```
# 9. Distribusi Periode Estimasi
periode_df = df_raw.groupBy("periode_estimasi") \
    .count() \
    .orderBy(desc("count"))

print("Distribusi data berdasarkan periode estimasi:")
periode_df.show(truncate=False)
```

Distribusi data berdasarkan periode estimasi:

periode_estimasi	count
Current Year	1654654
Next Year	1654654
Current Quarter	1654654
Next Quarter	1654654

Gambar 8 Distribusi Periode Estimasi

5.2.7 Top 10 Saham Berdasarkan Jumlah Data

Hasil EDA menunjukkan bahwa beberapa kode saham memiliki jumlah data yang sama, yaitu 1.716 record. Hal ini menunjukkan bahwa struktur data per saham cukup konsisten dan dapat digunakan untuk analisis berdasarkan kode saham.

```
# 11. Top 10 Kode Saham Berdasarkan Jumlah Data
top_saham_df = df_raw.groupBy("kode_saham") \
    .count() \
    .orderBy(desc("count")) \
    .limit(10)

print("Top 10 kode saham berdasarkan jumlah data:")
top_saham_df.show(truncate=False)
```

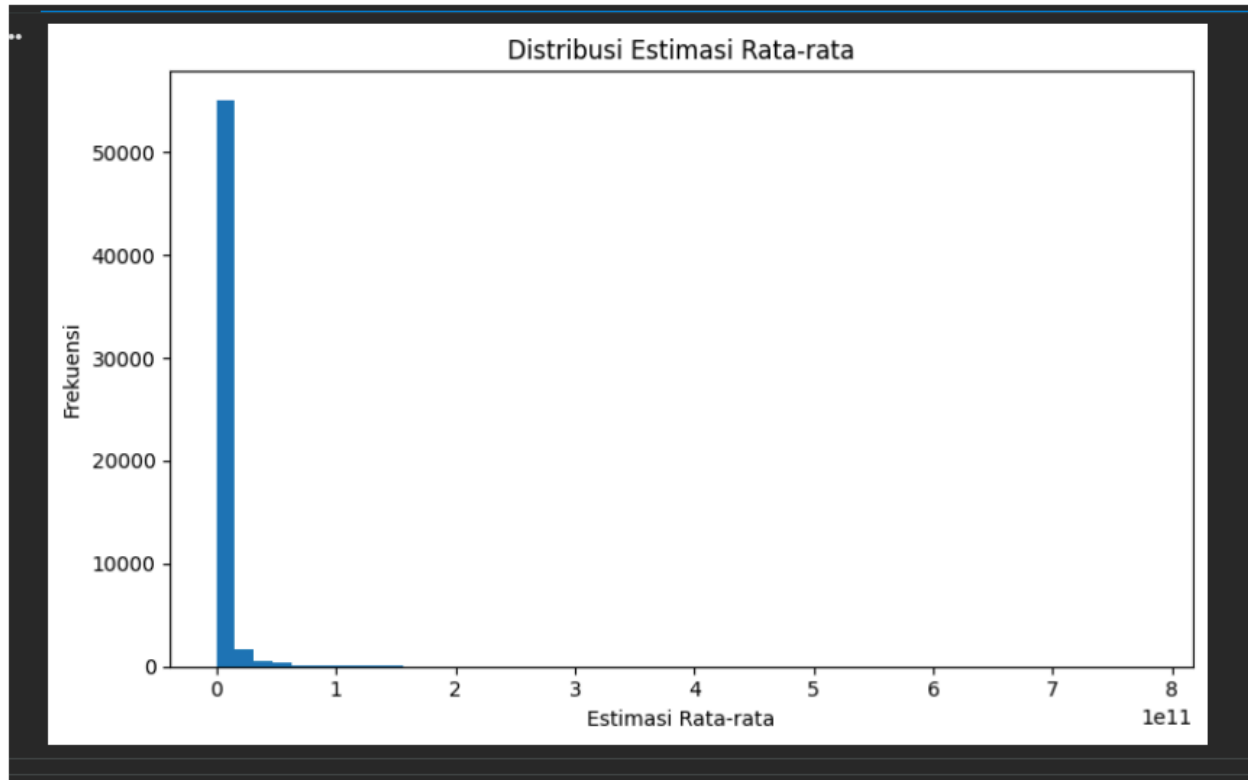
Top 10 kode saham berdasarkan jumlah data:

kode_saham	count
AMBA	1716
AFG	1716
ABM	1716
AGIO	1716
AER	1716
AAPL	1716
AGI	1716
ALLY	1716
ALGN	1716
ABEV	1716

Gambar 9 Top 10 Saham Berdasarkan Dataset

5.2.8 Histogram & Boxplot

Histogram `estimasi_rata_rata` menunjukkan bahwa sebagian besar data berada pada rentang nilai rendah hingga menengah, sedangkan sebagian kecil data memiliki nilai sangat tinggi. Pola ini menunjukkan distribusi yang condong ke kanan atau right-skewed.



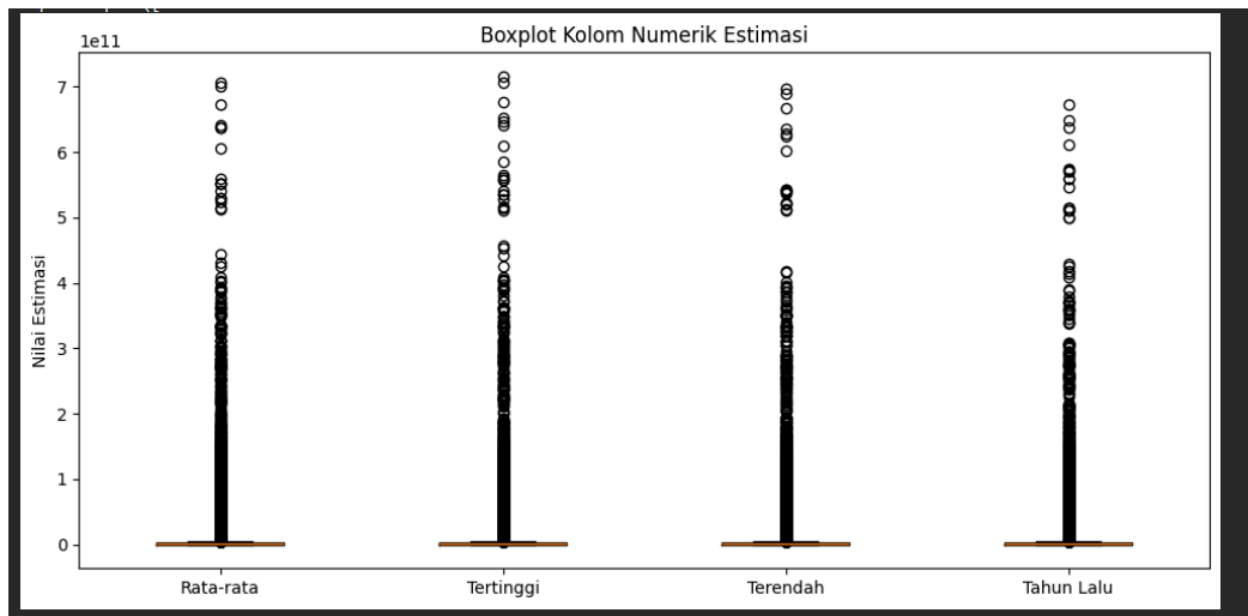
Gambar 10 Estimasi Rata-Rata

Gambar 11 Boxplot kolom numerik menunjukkan adanya banyak nilai ekstrem pada data estimasi. Namun, nilai ekstrem tidak langsung dihapus karena dalam konteks earnings dan sales estimate, perusahaan besar dapat memiliki nilai estimasi yang jauh lebih tinggi dibanding perusahaan lain.

```
# 14. Boxplot Kolom Numerik
sample_numeric_pd = df_raw.select(
    "estimasi_rata_rata",
    "estimasi_tertinggi",
    "estimasi_terendah",
    "estimasi_tahun_lalu"
).dropna() \
.sample(False, 0.01, seed=42) \
.toPandas()

plt.figure(figsize=(10, 5))
plt.boxplot([
    sample_numeric_pd["estimasi_rata_rata"],
    sample_numeric_pd["estimasi_tertinggi"],
    sample_numeric_pd["estimasi_terendah"],
    sample_numeric_pd["estimasi_tahun_lalu"]
], labels=[
    "Rata-rata",
    "Tertinggi",
    "Terendah",
    "Tahun Lalu"
])

plt.title("Boxplot Kolom Numerik Estimasi")
plt.ylabel("Nilai Estimasi")
plt.tight_layout()
plt.show()
```



Gambar 11 Boxplot Kolom Numerik

5.2 Data Cleaning dan Preprocessing

Berdasarkan hasil EDA, tidak ditemukan data duplikat dan tidak ditemukan nilai tidak logis pada `estimasi_tertinggi` maupun `jumlah_analis`. Oleh karena itu, tahap cleaning difokuskan pada penanganan missing value pada kolom utama yang dibutuhkan untuk analisis, yaitu `estimasi_rata_rata`, `jumlah_analis`, `estimasi_tertinggi`, dan `estimasi_terendah`. Kolom `estimasi_tahun_lalu` tidak digunakan sebagai syarat utama cleaning karena hanya diperlukan

untuk analisis pertumbuhan. Setelah proses cleaning, jumlah data berkurang dari 6.618.616 record menjadi 5.714.890 record, sehingga terdapat 903.726 record yang dihapus.

```
# 5.2 DATA CLEANING DAN PREPROCESSING

from pyspark.sql.functions import col, to_timestamp

def format_angka(nilai):
    return f"{nilai:,.}"

print("DATA CLEANING DAN PREPROCESSING")
|
# Jumlah data awal
jumlah_data_awal = df_raw.count()

# Kolom wajib untuk analisis utama
kolom_wajib_cleaning = [
    "Tanggal",
    "kode_saham",
    "periode_estimasi",
    "tanggal_akhir_periode",
    "estimasi_rata_rata",
    "jumlah_analis",
    "estimasi_tertinggi",
    "estimasi_terendah"
]

# Menghapus missing value pada kolom utama
df_clean = df_raw.dropna(subset=kolom_wajib_cleaning)
```

```
# Mengubah tipe data tanggal
df_clean = df_clean.withColumn(
    "Tanggal",
    to_timestamp(col("Tanggal"), "M/d/yy h:mm a")
)

df_clean = df_clean.withColumn(
    "tanggal_akhir_periode",
    to_timestamp(col("tanggal_akhir_periode"), "M/d/yy h:mm a")
)

# Mengubah jumlah_analis menjadi integer
df_clean = df_clean.withColumn(
    "jumlah_analis",
    col("jumlah_analis").cast("int")
)

# Ringkasan hasil cleaning
jumlah_data_bersih = df_clean.count()
jumlah_data_dihapus = jumlah_data_awal - jumlah_data_bersih

print("Jumlah data awal      :", format_angka(jumlah_data_awal))
print("Jumlah data setelah cleaning :", format_angka(jumlah_data_bersih))
print("Jumlah data yang dihapus   :", format_angka(jumlah_data_dihapus))

print("Schema setelah cleaning:")
df_clean.printSchema()

print("Contoh data setelah cleaning:")
df_clean.show(5, truncate=False)
```

```

DATA CLEANING DAN PREPROCESSING
Jumlah data awal      : 6.618.616
Jumlah data setelah cleaning : 5.714.890
Jumlah data yang dihapus : 903.726
Schema setelah cleaning:
root
|-- Tanggal: timestamp (nullable = true)
|-- kode_saham: string (nullable = true)
|-- periode_estimasi: string (nullable = true)
|-- tanggal_akhir_periode: timestamp (nullable = true)
|-- estimasi_rata_rata: double (nullable = true)
|-- jumlah_analisis: integer (nullable = true)
|-- estimasi_tertinggi: double (nullable = true)
|-- estimasi_terendah: double (nullable = true)
|-- estimasi_tahun_lalu: double (nullable = true)

Contoh data setelah cleaning:
+-----+-----+-----+-----+-----+-----+-----+-----+
|Tanggal|kode_saham|periode_estimasi|tanggal_akhir_periode|estimasi_rata_rata|jumlah_analisis|estimasi_tertinggi|estimasi_terendah|estimasi_tahun_lalu|
+-----+-----+-----+-----+-----+-----+-----+-----+
|2017-10-26 00:00:00|A|Current Quarter|2017-10-31 00:00:00|1.17E9|7|1.19E9|1.16E9|1.11E9|
|2017-10-26 00:00:00|A|Current Year|2017-10-31 00:00:00|4.45E9|8|4.47E9|4.44E9|4.2E9|
|2017-10-26 00:00:00|A|Next Quarter|2018-01-31 00:00:00|1.14E9|3|1.15E9|1.13E9|1.07E9|
|2017-10-26 00:00:00|A|Next Year|2018-10-31 00:00:00|4.72E9|8|4.76E9|4.68E9|4.45E9|
|2017-10-26 00:00:00|AA|Current Quarter|2017-12-31 00:00:00|3.37E9|2|3.44E9|3.29E9|2.54E9|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Gambar 12 Hasil Data Cleaning & Preprocessing

5.3 Data Processing Menggunakan Apache Spark

Setelah data melalui tahap cleaning dan preprocessing, tahap selanjutnya adalah data processing menggunakan Apache Spark. Tahap ini bertujuan untuk mengolah data bersih menjadi data hasil analisis yang siap digunakan pada tahap integrasi SQL, NoSQL, dan visualisasi dashboard. Data input yang digunakan pada tahap ini adalah df_clean dengan jumlah 5.714.890 record.

5.3.1 Pembuatan Kolom Analisis

Pada tahap processing, dilakukan penambahan dua kolom baru, yaitu rentang_estimasi dan selisih_dari_tahun_lalu. Kolom rentang_estimasi diperoleh dari selisih antara estimasi_tertinggi dan estimasi_terendah. Kolom ini digunakan untuk melihat tingkat stabilitas estimasi suatu saham. Semakin kecil nilai rentang estimasi, semakin kecil perbedaan antara estimasi tertinggi dan terendah. Sementara itu, kolom selisih_dari_tahun_lalu digunakan untuk melihat perubahan nilai estimasi dibandingkan dengan estimasi pada tahun sebelumnya.

```

# =====
# 5.3 DATA PROCESSING MENGGUNAKAN APACHE SPARK
# =====

from pyspark.sql.functions import col, avg, count, round, when, desc, asc
from IPython.display import display, Markdown
import pandas as pd

# Fungsi format angka agar output lebih rapi
def format_angka(nilai):
    return f"{nilai:,}".replace(",", ".")

display(Markdown("## 5.3 Data Processing Menggunakan Apache Spark"))

# Mengecek jumlah data input dari hasil cleaning
jumlah_data_input = df_clean.count()
print("Jumlah data input dari df_clean:", format_angka(jumlah_data_input))

# =====
# 1. Membuat DataFrame Hasil Processing
# =====

# Membuat kolom rentang_estimasi
# rentang_estimasi = estimasi_tertinggi - estimasi_terendah
# Digunakan untuk melihat stabilitas estimasi saham
df_processed = df_clean.withColumn(
    "rentang_estimasi",
    col("estimasi_tertinggi") - col("estimasi_terendah")
)

# Membuat kolom selisih_dari_tahun_lalu
# Digunakan sebagai dasar analisis pertumbuhan
df_processed = df_processed.withColumn(
    "selisih_dari_tahun_lalu",
    when(
        col("estimasi_tahun_lalu").isNotNull(),
        col("estimasi_rata_rata") - col("estimasi_tahun_lalu")
    ).otherwise(None)
)

print("Schema df_processed:")
df_processed.printSchema()

print("Contoh data df_processed:")
df_processed.show(5, truncate=False)

```

```

Jumlah data input dari df_clean: 5.714.890
Schema df_processed:
root
 |-- Tanggal: timestamp (nullable = true)
 |-- kode_saham: string (nullable = true)
 |-- periode_estimasi: string (nullable = true)
 |-- tanggal_akhir_periode: timestamp (nullable = true)
 |-- estimasi_rata_rata: double (nullable = true)
 |-- jumlah_analisis: integer (nullable = true)
 |-- estimasi_tertinggi: double (nullable = true)
 |-- estimasi_terendah: double (nullable = true)
 |-- estimasi_tahun_lalu: double (nullable = true)
 |-- rentang_estimasi: double (nullable = true)
 |-- selisih_dari_tahun_lalu: double (nullable = true)

Contoh data df_processed:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Tanggal|kode_saham|periode_estimasi|tanggal_akhir_periode|estimasi_rata_rata|jumlah_analisis|estimasi_tertinggi|estimasi_terendah|estimasi_tahun_lalu|rentang_estimasi|selisih_dari_tahun_lalu|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|2017-10-26 00:00:00|A|Current Quarter|2017-10-31 00:00:00|1.17E9|7|1.19E9|1.16E9|1.11E9|3.0E7|6.0E7|
|2017-10-26 00:00:00|A|Current Year|2017-10-31 00:00:00|4.45E9|8|4.47E9|4.44E9|4.2E9|2.5E8|2.5E8|
|2017-10-26 00:00:00|A|Next Quarter|2018-01-31 00:00:00|1.14E9|3|1.15E9|1.13E9|1.07E9|2.0E7|7.0E7|
|2017-10-26 00:00:00|A|Next Year|2018-10-31 00:00:00|4.72E9|8|4.76E9|4.68E9|4.45E9|8.0E7|2.7E8|
|2017-10-26 00:00:00|AA|Current Quarter|2017-12-31 00:00:00|3.37E9|2|3.44E9|3.29E9|2.54E9|1.5E8|8.3E8|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Gambar 13 Pembuatan Kolom Analisis

5.3.2 Ringkasan Berdasarkan Periode Estimasi

Hasil processing pertama adalah ringkasan data berdasarkan periode_estimasi. Berdasarkan hasil agregasi, data terbagi ke dalam empat periode, yaitu Current Year, Next Year, Current Quarter, dan Next Quarter. Periode Current Year memiliki jumlah data terbanyak, yaitu 1.486.202 record,

sedangkan Next Quarter memiliki jumlah data paling sedikit, yaitu 1.349.179 record. Dari sisi rata-rata estimasi, periode Next Year memiliki nilai rata-rata estimasi tertinggi, yaitu sekitar 7,52 miliar, sedangkan periode kuartalan memiliki nilai rata-rata estimasi yang lebih rendah.

```
# =====
# 2. Ringkasan Berdasarkan Periode Estimasi
# =====

period_summary = df_processed.groupby("periode_estimasi").agg(
    count("*").alias("jumlah_data"),
    round(avg("estimasi_rata_rata"), 2).alias("rata_rata_estimasi"),
    round(avg("jumlah_analis"), 2).alias("rata_rata_jumlah_analis"),
    round(avg("rentang_estimasi"), 2).alias("rata_rata_rentang_estimasi")
).orderBy(desc("jumlah_data"))

print("Ringkasan berdasarkan periode estimasi:")
period_summary.show(truncate=False)
```

```
Ringkasan berdasarkan periode estimasi:
+-----+-----+-----+-----+-----+
|periode_estimasi|jumlah_data|rata_rata_estimasi|rata_rata_jumlah_analis|rata_rata_rentang_estimasi|
+-----+-----+-----+-----+-----+
|Current Year    |1486202    |7.09273361018E9  |3.62                   |2.8920111211E8           |
|Next Year       |1483161    |7.5175760223E9  |3.6                    |5.6030972898E8           |
|Current Quarter |1396348    |1.71156082016E9 |3.42                   |9.231772469E7            |
|Next Quarter    |1349179    |1.75157272486E9 |3.25                   |1.1285597993E8           |
+-----+-----+-----+-----+-----+
```

Gambar 14 Ringkasan Berdasarkan Periode Estimasi

5.3.3 Analisis Saham Berdasarkan Estimasi Rata-Rata

Hasil processing berikutnya adalah analisis saham berdasarkan estimasi rata-rata tertinggi. Pada analisis ini, data dikelompokkan berdasarkan kode_saham, kemudian dihitung rata-rata nilai estimasi_rata_rata dan jumlah_analis. Hasilnya menunjukkan bahwa beberapa saham seperti SNP, WMT, PTR, AMZN, RDS.A, SHEL, BRK.B, AAPL, UNH, dan XOM termasuk dalam kelompok saham dengan rata-rata estimasi tertinggi. Hasil ini tidak dimaksudkan sebagai rekomendasi investasi, melainkan sebagai gambaran saham yang memiliki nilai estimasi paling tinggi berdasarkan data yang dianalisis.

```
# =====
# 3. Saham Berdasarkan Estimasi Rata-rata Tertinggi
# =====

# Data ini TIDAK dibatasi limit(10), sehingga semua hasil agregasi saham tetap disimpan.
# Untuk tampilan laporan, cukup gunakan show(10).
top_consensus = df_processed.groupby("kode_saham").agg(
    count("*").alias("jumlah_data"),
    round(avg("estimasi_rata_rata"), 2).alias("rata_rata_estimasi"),
    round(avg("jumlah_analis"), 2).alias("rata_rata_jumlah_analis")
).orderBy(desc("rata_rata_estimasi"))

print("Top 10 saham berdasarkan estimasi rata-rata tertinggi:")
top_consensus.show(10, truncate=False)
```

Top 10 saham berdasarkan estimasi rata-rata tertinggi:

kode_saham	jumlah_data	rata_rata_estimasi	rata_rata_jumlah_analis
SNP	500	4.0474042E11	1.11
WMT	1688	3.7415148104265E11	8.49
PTR	495	3.4968393939394E11	1.13
AMZN	1711	3.0600587375804E11	14.03
RDS.A	428	2.7216887850467E11	1.09
SHEL	592	2.4511640202703E11	1.17
BRK.B	1198	2.2943796327212E11	1.0
AAPL	1715	2.1974335860058E11	8.57
UNH	1695	2.0314772861357E11	8.3
XOM	1700	1.9331194117647E11	4.04

Gambar 15 Ringkasan Berdasarkan Periode Estimasi

5.3.4 Analisis Saham Berdasarkan Estimasi Paling Stabil

Selain itu, dilakukan analisis saham dengan estimasi paling stabil menggunakan kolom rentang_estimasi. Saham dengan nilai rata-rata rentang estimasi terkecil dianggap memiliki perbedaan yang rendah antara estimasi tertinggi dan estimasi terendah. Hasil ini menunjukkan beberapa saham dengan nilai rata-rata rentang estimasi 0, yang berarti estimasi tertinggi dan estimasi terendah pada data tersebut memiliki nilai yang sama. Namun, hasil ini tetap perlu dibaca dengan hati-hati karena beberapa saham memiliki jumlah data yang relatif kecil.

```
# =====
# 4. Saham dengan Estimasi Paling Stabil
# =====

# Data ini juga tidak dibatasi limit(10).
# Semua saham disimpan, tetapi yang ditampilkan hanya 10 teratas.
stable_stocks = df_processed.groupby("kode_saham").agg(
    count("*").alias("jumlah_data"),
    round(avg("estimasi_rata_rata"), 2).alias("rata_rata_estimasi"),
    round(avg("rentang_estimasi"), 2).alias("rata_rata_rentang_estimasi")
).orderBy(asc("rata_rata_rentang_estimasi"))

print("Top 10 saham dengan estimasi paling stabil:")
stable_stocks.show(10, truncate=False)
```

Top 10 saham dengan estimasi paling stabil:

kode_saham	jumlah_data	rata_rata_estimasi	rata_rata_rentang_estimasi
FTEK	1275	2.080752157E7	0.0
BKMU	56	6.5275E7	0.0
SMTX	472	2.4244169492E8	0.0
PKE	60	6.428483333E7	0.0
SBT	664	6.347108434E7	0.0
NRCG	163	2.9327423313E8	0.0
XGTI	28	2.48175E7	0.0
AAV	120	1.4445191667E8	0.0
PROS	355	5.3396901408E8	0.0
MNGA	4	7400000.0	0.0

Gambar 16 Analisis Saham Berdasarkan Estimasi Paling Stabil

5.3.5 Analisis Saham Berdasarkan Pertumbuhan Estimasi

Analisis terakhir adalah analisis pertumbuhan estimasi. Pada analisis ini, hanya data yang memiliki nilai `estimasi_tahun_lalu` dan nilainya lebih dari 0 yang digunakan agar perhitungan persentase pertumbuhan tetap valid. Dari 5.714.890 record pada `df_processed`, terdapat 5.442.639 record yang dapat digunakan untuk analisis pertumbuhan. Hasil agregasi menunjukkan saham dengan rata-rata pertumbuhan estimasi tertinggi berdasarkan perbandingan antara estimasi rata-rata saat ini dan estimasi tahun sebelumnya.

```

# =====
# 5. Saham Berdasarkan Pertumbuhan Estimasi Tertinggi
# =====

# Untuk analisis pertumbuhan, hanya digunakan data yang memiliki estimasi_tahun_lalu
# dan estimasi_tahun_lalu > 0 agar perhitungan persentase tidak salah.
df_growth_source = df_processed.filter(
    (col("estimasi_tahun_lalu").isNotNull()) &
    (col("estimasi_tahun_lalu") > 0)
)

df_growth_source = df_growth_source.withColumn(
    "pertumbuhan_persen",
    round(
        ((col("estimasi_rata_rata") - col("estimasi_tahun_lalu")) / col("estimasi_tahun_lalu")) * 100,
        2
    )
)

# Semua saham hasil agregasi pertumbuhan tetap disimpan.
growth_stocks = df_growth_source.groupBy("kode_saham").agg(
    count("*").alias("jumlah_data"),
    round(avg("estimasi_rata_rata"), 2).alias("rata_rata_estimasi"),
    round(avg("selisih_dari_tahun_lalu"), 2).alias("rata_rata_selisih_dari_tahun_lalu"),
    round(avg("pertumbuhan_persen"), 2).alias("rata_rata_pertumbuhan_persen")
).orderBy(desc("rata_rata_pertumbuhan_persen"))

print("Top 10 saham berdasarkan pertumbuhan estimasi tertinggi:")
growth_stocks.show(10, truncate=False)

```

```

Top 10 saham berdasarkan pertumbuhan estimasi tertinggi:
+-----+-----+-----+-----+-----+
|kode_saham|jumlah_data|rata_rata_estimasi|rata_rata_selisih_dari_tahun_lalu|rata_rata_pertumbuhan_persen|
+-----+-----+-----+-----+-----+
|FSR       |414        |7.0902504831E8   |5.8266753623E8                   |430842.83                   |
|VRAX      |24         |4.0E7            |3.992125E7                      |228025.0                    |
|CYAD      |364        |1.526656593E7   |8532170.33                      |60757.03                    |
|CRSP      |1656       |9.016269928E7   |-2.365745169E7                  |38997.76                    |
|ZAPP      |60         |1.377E8         |1.1969633333E8                  |37476.26                    |
|NEGG      |50         |1.54E9          |6.07312E8                       |35308.99                    |
|PHIO      |55         |6.773509091E7   |2.682036364E7                   |31874.03                    |
|ALT       |884        |5.398873303E7   |4.06543552E7                    |29944.01                    |
|SEV       |144        |4.137111111E7   |3.718958333E7                   |28779.71                    |
|OCGN      |326        |8.000159509E7   |4.614702454E7                   |28326.36                    |
+-----+-----+-----+-----+-----+

```

Gambar 17 Analisis Saham Berdasarkan Pertumbuhan Estimasi

5.3.6 Output Data Processing

Output utama dari tahap data processing adalah `df_processed`, `df_period`, `df_top`, `df_stable`, dan `df_growth`. `df_processed` merupakan data bersih yang telah ditambahkan kolom analisis. `df_period` berisi ringkasan berdasarkan periode estimasi. `df_top` berisi seluruh saham yang diurutkan berdasarkan estimasi rata-rata tertinggi. `df_stable` berisi seluruh saham yang diurutkan berdasarkan rentang estimasi terkecil. `df_growth` berisi seluruh saham yang diurutkan

berdasarkan pertumbuhan estimasi tertinggi. Output-output ini selanjutnya digunakan pada tahap integrasi database SQL dan NoSQL.

```
# =====
# 6. Konversi Hasil Processing ke Pandas untuk Integrasi MySQL
# =====

# Hasil agregasi ini relatif kecil dibanding data mentah,
# sehingga aman dikonversi ke Pandas untuk dimasukkan ke MySQL.
df_period = period_summary.toPandas()
df_top = top_consensus.toPandas()
df_stable = stable_stocks.toPandas()
df_growth = growth_stocks.toPandas()

print("Konversi hasil Spark ke Pandas berhasil.")
print("Jumlah baris df_period :", format_angka(len(df_period)))
print("Jumlah baris df_top      :", format_angka(len(df_top)))
print("Jumlah baris df_stable   :", format_angka(len(df_stable)))
print("Jumlah baris df_growth   :", format_angka(len(df_growth)))

# =====
# 7. Ringkasan Output Data Processing
# =====

print("RINGKASAN DATA PROCESSING")
print("Jumlah data df_processed      :", format_angka(df_processed.count()))
print("Jumlah data growth source      :", format_angka(df_growth_source.count()))
print("Jumlah baris df_period          :", format_angka(len(df_period)))
print("Jumlah baris df_top             :", format_angka(len(df_top)))
print("Jumlah baris df_stable          :", format_angka(len(df_stable)))
print("Jumlah baris df_growth          :", format_angka(len(df_growth)))

print("\nOutput yang dihasilkan:")
print("1. df_processed : data bersih yang sudah ditambah kolom analisis")
print("2. df_period    : ringkasan berdasarkan periode estimasi")
print("3. df_top       : semua saham yang diurutkan berdasarkan estimasi rata-rata tertinggi")
print("4. df_stable    : semua saham yang diurutkan berdasarkan rentang estimasi terkecil")
print("5. df_growth    : semua saham yang diurutkan berdasarkan pertumbuhan estimasi tertinggi")
```

```
RINGKASAN DATA PROCESSING
Jumlah data df_processed      : 5.714.890
Jumlah data growth source      : 5.442.639
Jumlah baris df_period        : 4
Jumlah baris df_top           : 6.436
Jumlah baris df_stable        : 6.436
Jumlah baris df_growth        : 6.325

Output yang dihasilkan:
1. df_processed : data bersih yang sudah ditambah kolom analisis
2. df_period    : ringkasan berdasarkan periode estimasi
3. df_top       : semua saham yang diurutkan berdasarkan estimasi rata-rata tertinggi
4. df_stable    : semua saham yang diurutkan berdasarkan rentang estimasi terkecil
5. df_growth    : semua saham yang diurutkan berdasarkan pertumbuhan estimasi tertinggi
```

Gambar 18 Output Hasil Processing

5.4 Integrasi Database SQL

Pada tahap ini, hasil pengolahan data yang telah diperoleh menggunakan Python disimpan ke dalam database SQL menggunakan koneksi MySQL. Proses ini bertujuan agar data hasil analisis dapat disimpan secara permanen, dikelola dengan lebih mudah, dan digunakan kembali untuk kebutuhan visualisasi maupun analisis lanjutan.

5.4.1 Konfigurasi Koneksi Database

```
DB_HOST='gateway01.ap-southeast-1.prod.alicloud.tidbcloud.com'
DB_PORT=4000
DB_USERNAME='47rWkweNDcDdekz.root'
DB_PASSWORD='*****'
DB_DATABASE='test'

def create_db_connection(host_name, user_name, user_password, db_name):
    connection = None
    try:
        connection = mysql.connector.connect(
            host=host_name,
            user=user_name,
            passwd=user_password,
            database=db_name,
            port=DB_PORT # Added port parameter
        )
        print("MySQL Database connection successful")
    except Error as err:
        print(f"Error: '{err}'")
    return connection
```

Gambar 19 Konfigurasi Koneksi Database

Pada Gambar 19 dilakukan konfigurasi parameter koneksi yang terdiri dari alamat host, port, username, password, dan nama database. Informasi tersebut digunakan sebagai identitas untuk menghubungkan aplikasi Python dengan database TiDB Cloud. Fungsi `create_db_connection()` bertugas membangun koneksi ke database menggunakan library `mysql.connector`. Proses koneksi dilakukan melalui metode `mysql.connector.connect()` dengan memanfaatkan parameter yang

telah didefinisikan sebelumnya. Jika koneksi berhasil dilakukan, sistem akan menampilkan pesan keberhasilan. Sebaliknya, jika terjadi kesalahan seperti kesalahan autentikasi atau server tidak dapat diakses, sistem akan menangkap exception dan menampilkan informasi error yang terjadi.

5.4.2 Pembuatan Tabel Secara Otomatis

```
table_name = table_name.lower().replace(' ', '_').replace('-', '_')

# Drop table if it exists to ensure a clean insert
drop_table_query = f"DROP TABLE IF EXISTS {table_name}"
execute_query(connection, drop_table_query)

# Create table based on DataFrame schema
create_table_query_parts = []
for col, dtype in df_pandas.dtypes.items():
    sql_type = "TEXT"
    if 'int' in str(dtype):
        sql_type = "BIGINT"
    elif 'float' in str(dtype) or 'double' in str(dtype):
        sql_type = "DOUBLE"
    elif 'datetime' in str(dtype):
        sql_type = "DATETIME"
    create_table_query_parts.append(f"`{col}` {sql_type}")
create_table_query = f"CREATE TABLE {table_name} ({', '.join(create_table_query_parts)})"
execute_query(connection, create_table_query)
```

Gambar 20 Pembuatan Tabel Secara Otomatis

Pada Gambar 20 sistem melakukan normalisasi nama tabel agar sesuai dengan aturan penamaan pada database. Proses ini dilakukan dengan mengubah seluruh huruf menjadi huruf kecil (lowercase) serta mengganti spasi dan tanda hubung menjadi karakter garis bawah (underscore). Sebelum membuat tabel baru, sistem terlebih dahulu menjalankan perintah DROP TABLE IF EXISTS untuk menghapus tabel lama apabila tabel dengan nama yang sama telah tersedia. Langkah ini bertujuan untuk memastikan data yang tersimpan merupakan hasil analisis terbaru. Selanjutnya dilakukan proses identifikasi tipe data setiap kolom pada DataFrame menggunakan atribut dtypes. Setiap tipe data Pandas kemudian dipetakan ke tipe data SQL yang sesuai. Data bertipe integer dikonversi menjadi BIGINT, data numerik desimal menjadi DOUBLE, data waktu menjadi DATETIME, sedangkan tipe data lainnya disimpan sebagai TEXT. Setelah seluruh kolom berhasil dipetakan, sistem membentuk perintah SQL CREATE TABLE secara dinamis dan mengeksekusinya pada database. Dengan pendekatan ini, struktur tabel dapat dibuat secara otomatis tanpa perlu mendefinisikan skema tabel secara manual.

5.4.3 Penyimpanan Data ke Database

```
# Prepare insert query
cols = ", ".join([f"`{col}`" for col in df_pandas.columns])
placeholders = ", ".join(["%s"] * len(df_pandas.columns))
insert_query = f"INSERT INTO {table_name} ({cols}) VALUES ({placeholders})"

cursor = connection.cursor()
data_to_insert = [tuple(row) for row in df_pandas.values]
try:
    cursor.executemany(insert_query, data_to_insert)
    connection.commit()
    print(f"Successfully inserted {len(df_pandas)} rows into MySQL table '{table_name}'")
except Error as err:
    print(f"Error inserting data into {table_name}: '{err}'")
```

Gambar 21 Penyimpanan Data ke Database

Pada Gambar 21 dilakukan proses penyimpanan data dari DataFrame ke dalam tabel database. Sistem terlebih dahulu membentuk daftar nama kolom dan placeholder yang akan digunakan dalam perintah INSERT INTO. Data yang terdapat pada DataFrame kemudian dikonversi menjadi kumpulan tuple menggunakan `df_pandas.values`. Format ini diperlukan agar data dapat diproses oleh metode `executemany()`. Metode `executemany()` digunakan untuk melakukan penyisipan banyak data sekaligus dalam satu proses eksekusi. Pendekatan ini lebih efisien dibandingkan melakukan perintah INSERT secara berulang untuk setiap baris data. Setelah seluruh data berhasil disimpan, metode `commit()` dipanggil untuk memastikan perubahan tersimpan secara permanen pada database.

5.4.4 Penyimpanan Data ke Database

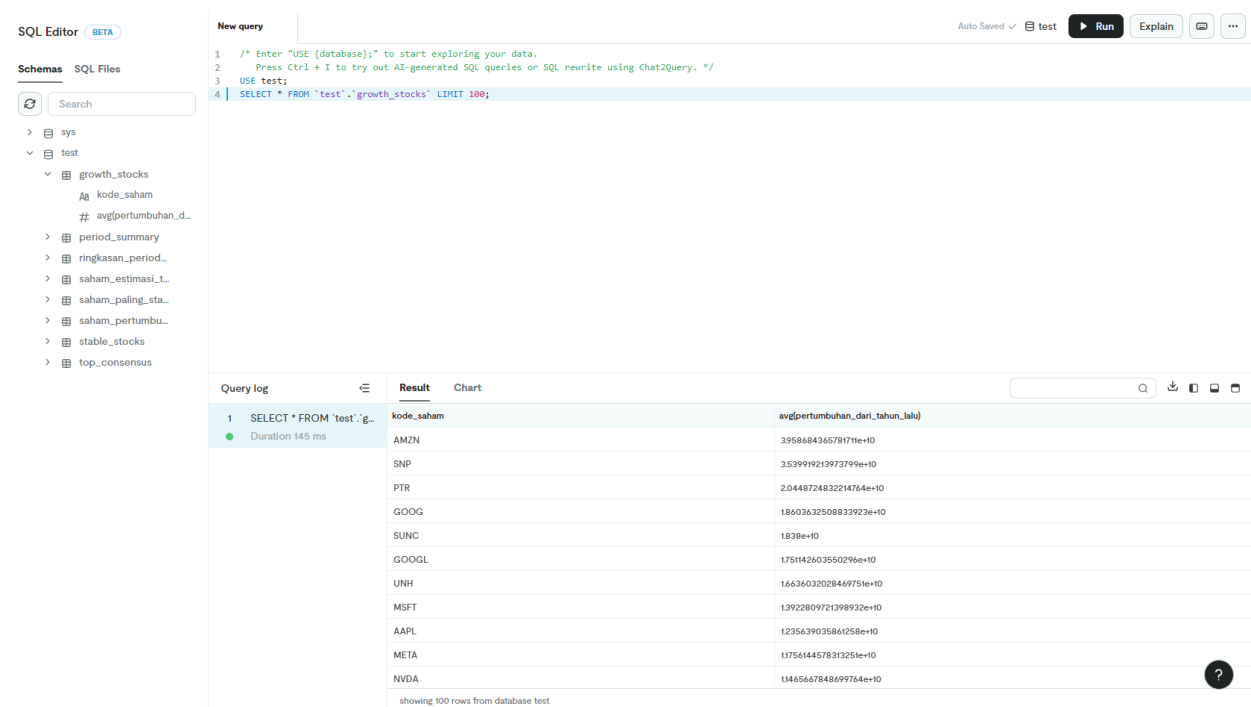
```
# Establish connection
mysql_connection = create_db_connection(DB_HOST, DB_USERNAME, DB_PASSWORD, DB_DATABASE)

if mysql_connection:
    # Store each processed DataFrame into MySQL
    print("\n--- Storing dataframes to MySQL ---")
    insert_df_to_mysql(df_period, "ringkasan_periode_estimasi", mysql_connection)
    insert_df_to_mysql(df_top, "saham_estimasi_tertinggi", mysql_connection)
    insert_df_to_mysql(df_stable, "saham_paling_stabil", mysql_connection)
    insert_df_to_mysql(df_growth, "saham_pertumbuhan_tertinggi", mysql_connection)
    mysql_connection.close()
    print("MySQL connection closed.")
else:
    print("Skipping MySQL integration due to connection failure.")
```

Gambar 22 Penyimpanan Data ke Database

Pada Gambar 22 dilakukan proses penyimpanan seluruh hasil analisis ke dalam database. Sistem terlebih dahulu membuat koneksi menggunakan fungsi `create_db_connection()`. Setelah koneksi berhasil dibuat, fungsi `insert_df_to_mysql()` dipanggil untuk menyimpan masing-masing DataFrame ke tabel yang berbeda sesuai dengan kategori analisis yang dihasilkan. Empat tabel yang disimpan meliputi tabel ringkasan periode estimasi, tabel saham dengan estimasi pendapatan tertinggi, tabel saham paling stabil, dan tabel saham dengan pertumbuhan pendapatan tertinggi. Setelah seluruh proses penyimpanan selesai dilakukan, koneksi database ditutup menggunakan metode `close()` untuk menghindari penggunaan sumber daya yang tidak diperlukan serta menjaga keamanan koneksi database.

5.4.5 Hasil Integrasi dan Penyimpanan Data



Gambar 23 Hasil Integrasi dan Penyimpanan Data

Gambar 23 menunjukkan hasil akhir dari proses integrasi data antara aplikasi Python dan database SQL yang dijalankan pada layanan TiDB Cloud. Setelah proses analisis selesai dilakukan, seluruh DataFrame hasil pengolahan data berhasil disimpan ke dalam database dan dapat diakses melalui SQL Editor. Pada panel Schemas terlihat beberapa tabel yang berhasil dibuat secara otomatis oleh sistem, yaitu `growth_stocks`, `period_summary`, `ringkasan_periode_estimasi`, `saham_estimasi_tertinggi`, `saham_paling_stabil`,

saham_pertumbuhan_tertinggi, stable_stocks, dan top_consensus. Tabel-tabel tersebut merupakan representasi dari hasil analisis yang sebelumnya disimpan dalam bentuk Pandas DataFrame.

5.5 Integrasi NoSQL

Pada tahap ini, data yang sebelumnya telah disimpan pada database SQL (TiDB Cloud) diintegrasikan ke database NoSQL menggunakan MongoDB Atlas. Proses ini bertujuan untuk menyediakan alternatif penyimpanan data yang lebih fleksibel, terutama untuk pengelolaan data dalam format dokumen (document-oriented database). Integrasi dilakukan dengan mengambil data dari tabel-tabel yang terdapat pada database SQL, kemudian mengubahnya menjadi dokumen JSON dan menyimpannya ke dalam koleksi (collection) MongoDB.

5.5.1 Koneksi Database SQL dan MongoDB

```
MONGO_URI = "mongodbsrv://E5:qweso@cluster0.gylgoi4.mongodb.net/?appName=Cluster0&tlsAllowInvalidCertificates=true"
MONGO_DB_NAME = 'bigdata'

DB_HOST='gateway01.ap-southeast-1.prod.alicloud.tidbcloud.com'
DB_PORT=4000
DB_USERNAME='47rWkweNDcDdekz.root'
DB_PASSWORD='*****'
DB_DATABASE='test'
```

```
def create_mysql_connection_for_fetch(host_name, user_name, user_password, db_name, port):
    connection = None
    try:
        connection = mysql.connector.connect(
            host=host_name,
            user=user_name,
            passwd=user_password,
            database=db_name,
            port=port
        )
        print("MySQL Database connection successful for fetching data.")
    except Error as err:
        print(f"Error establishing MySQL connection for fetching: '{err}'")
    return connection
```

```
def connect_to_mongodb(uri, db_name):
    try:
        client = MongoClient(uri)
        client.admin.command('ping') |
        db = client[db_name]
        print(f"MongoDB connection successful to database '{db_name}'")
        return client, db
    except ConnectionFailure as e:
        print(f"MongoDB connection failed: {e}")
        return None, None
```

Gambar 24 Koneksi Database SQL dan MongoDB

Gambar 24 menunjukkan proses konfigurasi koneksi ke dua sistem database yang berbeda, yaitu TiDB Cloud sebagai database SQL dan MongoDB Atlas sebagai database NoSQL. Koneksi ke TiDB Cloud digunakan untuk mengambil data hasil analisis yang telah disimpan pada tahap sebelumnya, sedangkan koneksi ke MongoDB digunakan sebagai tujuan penyimpanan data. Fungsi `create_mysql_connection_for_fetch()` bertugas membangun koneksi ke database SQL menggunakan library `mysql.connector`. Sementara itu, fungsi `connect_to_mongodb()` menggunakan library `pymongo` untuk membangun koneksi ke MongoDB Atlas. Sebelum koneksi digunakan, sistem menjalankan perintah ping untuk memastikan server MongoDB dapat diakses dengan baik.

5.5.2 Pengambilan Data dari Database SQL

```
def fetch_data_from_mysql_to_df(mysql_conn, table_name):
    df = pd.DataFrame()
    if mysql_conn is None:
        print("MySQL connection is not established. Cannot fetch data.")
        return df
    try:
        cursor = mysql_conn.cursor(dictionary=True)
        query = f"SELECT * FROM `{table_name}`"
        cursor.execute(query)
        records = cursor.fetchall()
        if records:
            df = pd.DataFrame(records)
            print(f"Successfully fetched {len(df)} rows from MySQL table '{table_name}'.")
        else:
            print(f"No records found in MySQL table '{table_name}'.")
    except Error as err:
        print(f"Error fetching data from MySQL table '{table_name}': '{err}'")
    finally:
        if cursor:
            cursor.close()
    return df
```

Gambar 25 Pengambilan Data dari Database SQL

Pada Gambar 25 dilakukan proses pengambilan data dari database SQL. Sistem menjalankan perintah `SELECT *` untuk membaca seluruh data yang terdapat pada tabel yang dipilih. Data yang diperoleh dari database kemudian dikonversi menjadi Pandas DataFrame agar dapat diproses lebih lanjut menggunakan Python. Penggunaan parameter `dictionary=True` memungkinkan setiap baris data dikembalikan dalam bentuk pasangan key-value, sehingga proses konversi ke DataFrame menjadi lebih mudah dilakukan. Dengan pendekatan ini, data yang tersimpan pada TiDB Cloud dapat digunakan kembali sebagai sumber data untuk integrasi ke MongoDB.

5.5.3 Penyimpanan Data ke MongoDB

```
def insert_df_to_mongodb(df_to_insert, collection_name, db):
    collection = db[collection_name]
    # Ensure DataFrame is not empty
    if df_to_insert.empty:
        print(f"DataFrame for '{collection_name}' is empty. Skipping MongoDB insertion.")
        return

    data_to_insert = df_to_insert.to_dict(orient='records')

    try:
        # Clear existing data in collection before inserting new data
        collection.delete_many({})
        if data_to_insert:
            collection.insert_many(data_to_insert)
            print(f"Successfully inserted {len(data_to_insert)} documents into MongoDB collection '{collection_name}'")
        else:
            print(f"No data to insert into MongoDB collection '{collection_name}'")
    except Exception as e:
        print(f"Error inserting data into {collection_name}: {e}")
```

Gambar 26 Penyimpanan Data ke MongoDB

Pada Gambar.26 dilakukan proses penyimpanan data ke MongoDB. Data yang sebelumnya berada dalam bentuk DataFrame dikonversi menjadi format list of dictionaries menggunakan metode `to_dict(orient='records')`. Format tersebut sesuai dengan struktur dokumen JSON yang digunakan oleh MongoDB. Sebelum data baru dimasukkan, sistem terlebih dahulu menghapus seluruh dokumen lama menggunakan perintah `delete_many({})` agar tidak terjadi duplikasi data. Setelah proses pembersihan selesai, data dimasukkan ke dalam koleksi MongoDB menggunakan metode `insert_many()`. Metode ini memungkinkan banyak dokumen disimpan sekaligus dalam satu operasi sehingga proses menjadi lebih efisien.

5.5.4 Migrasi Data dari SQL ke NoSQL

```
table_mappings = {
    "ringkasan_periode_estimasi": "ringkasan_periode_estimasi",
    "saham_estimasi_tertinggi": "saham_estimasi_tertinggi",
    "saham_paling_stabil": "saham_paling_stabil",
    "saham_pertumbuhan_tertinggi": "saham_pertumbuhan_tertinggi"
}

for mysql_table, mongo_collection in table_mappings.items():
    df_from_mysql = fetch_data_from_mysql_to_df(mysql_fetch_connection, mysql_table)
    insert_df_to_mongodb(df_from_mysql, mongo_collection, mongo_db)
```

Gambar 27 Migrasi Data dari SQL ke NoSQL

Gambar 27 menunjukkan proses migrasi data dari database SQL ke database NoSQL. Sistem menggunakan struktur `table_mappings` untuk memetakan nama tabel pada TiDB Cloud dengan

nama koleksi pada MongoDB. Untuk setiap tabel yang terdapat pada pemetaan, sistem akan mengambil data menggunakan fungsi `fetch_data_from_mysql_to_df()` kemudian menyimpannya ke MongoDB menggunakan fungsi `insert_df_to_mongodb()`. Melalui mekanisme ini, seluruh hasil analisis yang sebelumnya tersimpan pada database relasional dapat direplikasi ke database dokumen secara otomatis tanpa memerlukan proses input manual.

5.5.4 Migrasi Data dari SQL ke NoSQL

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
analisis_potensi_saham_lanjutan	-	835.58 kB	1.21 MB	6.8K	176.00 B	1	278.53 kB
analisis_potensi_saham_lanjutan_clean	-	815.10 kB	1.14 MB	6.4K	176.00 B	1	327.68 kB
analisis_risiko_dan_growth	-	819.20 kB	700.33 kB	6.8K	102.00 B	1	552.96 kB
growth_stocks	-	606.21 kB	536.11 kB	6.3K	84.00 B	1	446.46 kB
period_summary	-	36.86 kB	288.00 B	4	72.00 B	1	36.86 kB
ringkasan_periode_estimasi	-	36.86 kB	272.00 B	4	68.00 B	1	36.86 kB
saham	-	4.10 kB	0 B	0	0 B	1	4.10 kB
saham_estimasi_tertinggi	-	606.21 kB	486.59 kB	6.4K	75.00 B	1	516.10 kB
saham_paling_stabil	-	630.78 kB	387.45 kB	5.3K	73.00 B	1	430.08 kB
saham_pertumbuhan_tertinggi	-	577.54 kB	536.11 kB	6.3K	84.00 B	1	389.12 kB
stable_stocks	-	364.54 kB	387.45 kB	5.3K	73.00 B	1	245.76 kB
top_consensus	-	661.15 kB	486.59 kB	6.4K	75.00 B	1	528.38 kB

Gambar 28 Migrasi Data dari SQL ke NoSQL

Gambar 28 menunjukkan hasil akhir proses integrasi data ke MongoDB Atlas. Terlihat bahwa beberapa koleksi berhasil dibuat sesuai dengan tabel yang terdapat pada database SQL, yaitu `ringkasan_periode_estimasi`, `saham_estimasi_tertinggi`, `saham_paling_stabil`, dan `saham_pertumbuhan_tertinggi`. Setiap koleksi menyimpan data dalam bentuk dokumen JSON yang berisi pasangan atribut dan nilai (key-value pairs). Struktur penyimpanan ini memberikan fleksibilitas yang lebih tinggi dibandingkan database relasional karena tidak memerlukan skema tabel yang kaku. Keberhasilan munculnya dokumen pada masing-masing koleksi menunjukkan bahwa proses migrasi data dari TiDB Cloud ke MongoDB Atlas telah berjalan dengan baik. Dengan demikian, data hasil analisis tersedia pada dua jenis sistem database yang berbeda, yaitu SQL dan NoSQL, sehingga dapat mendukung berbagai kebutuhan pengelolaan dan analisis data di masa mendatang.

6. Hasil dan Analisis

6.1 Analisis Distribusi Data Berdasarkan Periode Estimasi

Analisis data yang dilakukan pada Gambar 29 digunakan untuk menganalisis distribusi data berdasarkan periode estimasi dengan cara mengidentifikasi kolom periode estimasi dan kolom jumlah data, meskipun nama kolom nya berbeda-beda. Setelah kolom ditemukan, data diurutkan berdasarkan jumlah data secara menurun sehingga periode dengan jumlah data terbesar di urutan teratas. Proses ini menghasilkan output yang menunjukkan bahwa Tahun Berjalan memiliki jumlah data paling banyak yaitu 1.486.202 data, diikuti oleh Tahun Berikutnya, Kuartal Berjalan, dan Kuartal Berikutnya. Berdasarkan hasil urutan tersebut, kode kemudian secara otomatis mengambil baris teratas dan menampilkan insight bahwa periode estimasi dengan jumlah data terbanyak adalah Tahun Berjalan, yang menandakan periode ini paling dominan dan paling sering digunakan dalam dataset.

```
=== Analisis Distribusi Periode Estimasi ===
```

	periode_estimasi	jumlah_data	rata_rata_estimasi	rata_rata_jumlah_analis	rata_rata_rentang_estimasi
0	Current Year	1486202	7.092734e+09	3.62	2.892011e+08
1	Next Year	1483161	7.517576e+09	3.60	5.603097e+08
2	Current Quarter	1396348	1.711561e+09	3.42	9.231772e+07
3	Next Quarter	1349179	1.751573e+09	3.25	1.128560e+08

Insight: Periode estimasi dengan jumlah data terbanyak adalah Current Year dengan 1.486.202 data.

Gambar 29 Analisis Distribusi Data Berdasarkan Periode Estimasi

6.2 Analisis Saham dengan Estimasi Rata-rata tertinggi

Kode pada Gambar 30 digunakan untuk mengidentifikasi 10 saham dengan nilai estimasi rata-rata tertinggi berdasarkan data yang tersedia. Pertama, kode mencari kolom yang berisi kode saham dan nilai estimasi rata-rata, meskipun nama kolomnya berbeda-beda. Setelah kolom ditemukan, data kemudian diurutkan secara menurun berdasarkan nilai estimasi rata-rata, sehingga saham dengan estimasi tertinggi berada di urutan teratas, lalu diambil 10 saham teratas. Proses ini menghasilkan output yang menampilkan saham seperti SNP, WMT, PTR, AMZN, hingga XOM sebagai saham dengan estimasi rata-rata tertinggi. Dari hasil urutan tersebut, kode secara otomatis mengambil saham pada baris pertama, yaitu SNP, dan menampilkan insight bahwa SNP merupakan saham dengan estimasi rata-rata tertinggi, dengan nilai estimasi sekitar 404,7 miliar, sehingga saham ini dianggap memiliki estimasi nilai paling tinggi dibanding saham lainnya dalam dataset.

=== Top 10 Saham dengan Estimasi Rata-rata Tertinggi ===

	kode_saham	jumlah_data	rata_rata_estimasi	rata_rata_jumlah_analis
0	SNP	500	4.047404e+11	1.11
1	WMT	1688	3.741515e+11	8.49
2	PTR	495	3.496839e+11	1.13
3	AMZN	1711	3.060059e+11	14.03
4	RDS.A	428	2.721689e+11	1.09
5	SHEL	592	2.451164e+11	1.17
6	BRK.B	1198	2.294380e+11	1.00
7	AAPL	1715	2.197434e+11	8.57
8	UNH	1695	2.031477e+11	8.30
9	XOM	1700	1.933119e+11	4.04

Insight: Saham dengan estimasi rata-rata tertinggi adalah SNP dengan nilai estimasi 404,740,420,000.00.

Gambar 30 Analisis Saham dengan Estimasi Rata-rata tertinggi

6.3 Analisis Saham dengan Estimasi Paling Stabil

Pada Gambar 31 merupakan Analisis stabilitas dilakukan berdasarkan nilai rata-rata rentang estimasi. Hasil analisis menunjukkan bahwa beberapa saham memiliki rata-rata rentang estimasi sebesar 0. Nilai tersebut menunjukkan bahwa estimasi tertinggi dan estimasi terendah memiliki nilai yang sama pada data tersebut. Namun, hasil ini perlu dibaca secara hati-hati karena beberapa saham memiliki jumlah data yang relatif kecil. Selain itu, istilah stabilitas pada analisis ini mengacu pada stabilitas estimasi, bukan stabilitas harga saham

=== Top 10 Saham dengan Estimasi Paling Stabil ===

	kode_saham	jumlah_data	rata_rata_estimasi	rata_rata_rentang_estimasi
822	VLD	418	6.773010e+07	0.0
845	LMDX	408	1.429049e+08	0.0
846	PRSO	696	1.124447e+07	0.0
847	OSA	135	1.918141e+07	0.0
816	RVAC	10	1.010200e+09	0.0
817	CASS	620	1.282753e+08	0.0
818	IXHL	169	2.349704e+06	0.0
819	COMS	207	3.281845e+07	0.0
820	VAXX	150	9.481333e+06	0.0
821	AREN	367	1.722635e+08	0.0

Insight: Saham dengan estimasi paling stabil adalah VLD karena memiliki rata-rata rentang estimasi paling kecil, yaitu 0.00.

Gambar 31 Analisis Saham dengan Estimasi Paling Stabil

6.4 Analisis Top 10 Saham dengan Pertumbuhan Estimasi Tertinggi

Analisis pertumbuhan pada Gambar 32 dilakukan menggunakan persentase pertumbuhan agar perbandingan antar saham lebih proporsional. Hasil analisis menunjukkan bahwa FSR memiliki

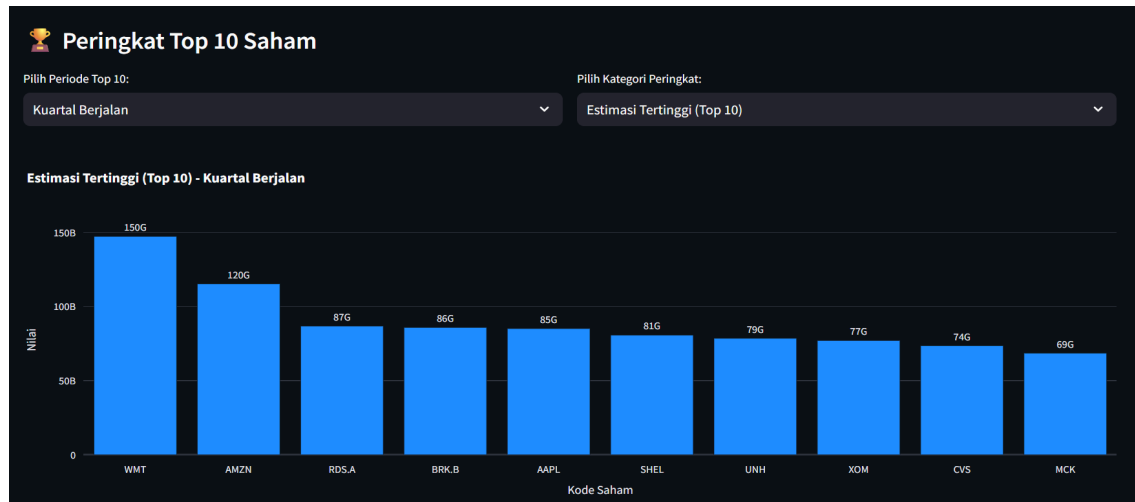
rata-rata pertumbuhan estimasi tertinggi, yaitu 430.842,83%. Hasil ini berbeda dari analisis selisih nominal karena analisis terbaru menggunakan persentase pertumbuhan, bukan hanya nilai selisih antara estimasi saat ini dan estimasi tahun sebelumnya. Hasil ini bukan rekomendasi investasi, tetapi insight awal dari data estimasi.

=== Top 10 Saham dengan Pertumbuhan Estimasi Tertinggi ===					
	kode_saham	jumlah_data	rata_rata_estimasi	rata_rata_selisih_dari_tahun_lalu	rata_rata_pertumbuhan_persen
0	FSR	414	7.090250e+08	5.826675e+08	430842.83
1	VRAX	24	4.000000e+07	3.992125e+07	228025.00
2	CYAD	364	1.526657e+07	8.532170e+06	60757.03
3	CRSP	1656	9.016270e+07	-2.365745e+07	38997.76
4	ZAPP	60	1.377000e+08	1.196963e+08	37476.26
5	NEGG	50	1.540000e+09	6.073120e+08	35308.99
6	PHIO	55	6.773509e+07	2.682036e+07	31874.03
7	ALT	884	5.398873e+07	4.065436e+07	29944.01
8	SEV	144	4.137111e+07	3.718958e+07	28779.71
9	OCGN	326	8.000160e+07	4.614702e+07	28326.36
Insight: Saham dengan pertumbuhan estimasi tertinggi adalah FSR dengan rata-rata pertumbuhan 430,842.83%.					

Gambar 32 Analisis Top 10 Saham dengan Pertumbuhan Estimasi Tertinggi

6.5 Analisis 10 Saham Estimasi Tertinggi pada Periode Kuartal Berjalan

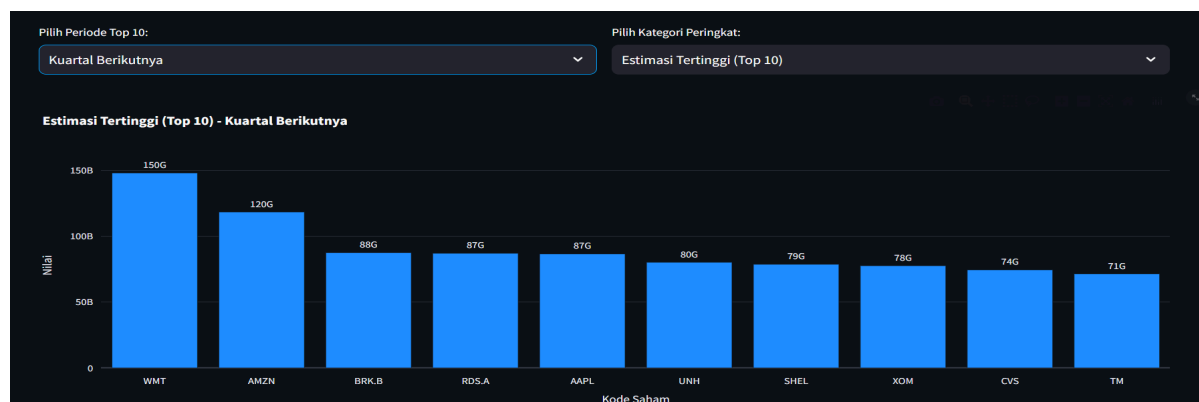
Pada Gambar 33 visualisasi menampilkan daftar 10 perusahaan dengan kapitalisasi pasar raksasa dunia yang diproyeksikan memiliki angka estimasi tertinggi oleh para analis pada kuartal ini. Berdasarkan data, WMT memimpin di urutan pertama dengan nilai estimasi mencapai sekitar 150 Miliar, disusul oleh AMZN di kisaran 120 Miliar, dan RDS.A di angka 87 Miliar, bersama dengan perusahaan *blue-chip* lainnya seperti BRK.B dan AAPL. Karena perusahaan-perusahaan ini konsisten mencatatkan kinerja fundamental yang positif, mereka mendapatkan penilaian analitis yang tinggi. Secara objektif, grafik ini menyajikan referensi saham berisiko rendah (*low-risk*) yang ideal bagi investor yang memprioritaskan kepastian pendapatan dan stabilitas modal di tengah dinamika pasar.



Gambar 33 Saham Estimasi Tertinggi pada Periode Kuartal Berjalan

6.6 Analisis 10 Saham Estimasi Tertinggi pada Periode Kuartal Berikutnya

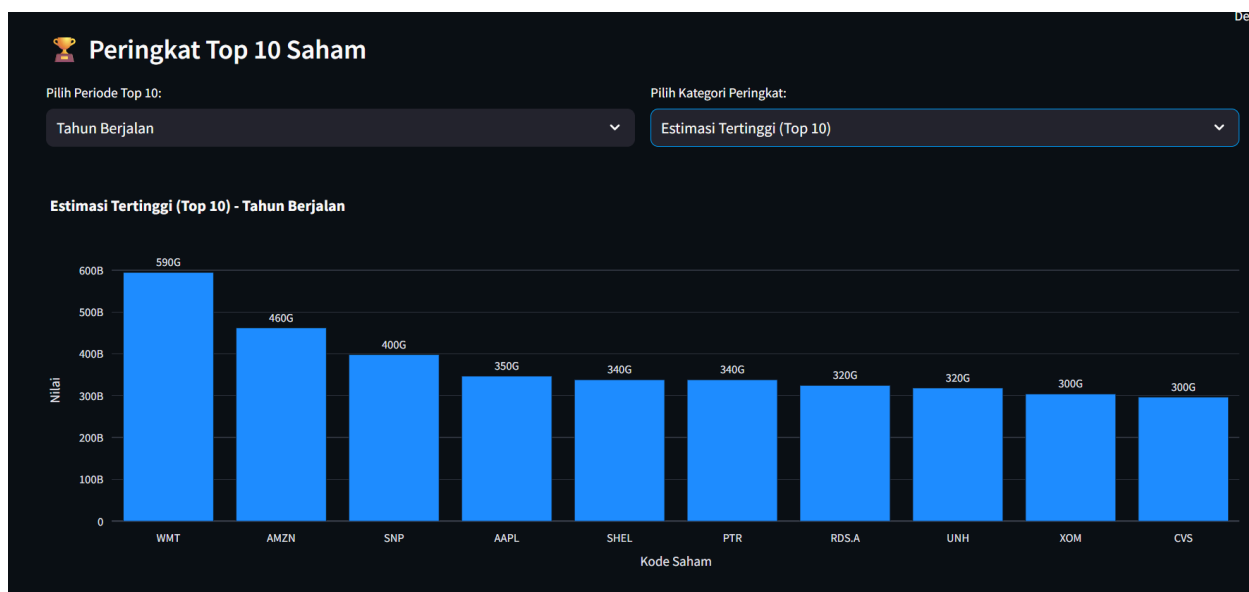
Visualisasi ini menampilkan daftar 10 perusahaan dengan kapitalisasi pasar raksasa dunia yang diproyeksikan memiliki angka estimasi tertinggi oleh para analis untuk kuartal mendatang. Berdasarkan data prospektif ini, WMT tetap memimpin di urutan pertama dengan nilai estimasi sekitar 150 Miliar, disusul oleh AMZN di kisaran 120 Miliar. Posisi ketiga ditempati oleh BRK.B di angka 88 Miliar, yang bersaing sangat ketat dengan RDS.A dan AAPL di kisaran 87 Miliar. Karena perusahaan-perusahaan ini memiliki fundamental kuat dan konsisten mencatatkan kinerja positif, mereka mendapatkan penilaian analisis prospektif yang tinggi. Secara objektif, grafik ini menyajikan referensi saham berisiko rendah (*low-risk*) yang ideal bagi investor yang memprioritaskan stabilitas modal dan kepastian pendapatan untuk periode investasi ke depan.



Gambar 34 Analisis 10 Saham Estimasi Tertinggi pada Periode Kuartal Berikutnya

6.7 Analisis 10 Saham Estimasi Tertinggi pada Periode Tahun Berjalan

Visualisasi ini menampilkan daftar 10 perusahaan dengan kapitalisasi pasar raksasa dunia yang diproyeksikan memiliki angka estimasi tertinggi oleh para analis sepanjang tahun ini. Berdasarkan data tahunan tersebut, WMT memimpin dengan nilai estimasi yang sangat masif, yaitu sekitar 590 Miliar. Posisi ini disusul oleh AMZN di kisaran 460 Miliar, dan SNP yang menempati urutan ketiga di angka 400 Miliar. Daftar ini juga dilengkapi oleh perusahaan-perusahaan berfundamental kuat lainnya seperti AAPL, SHEL, dan PTR. Karena perusahaan-perusahaan ini konsisten mencatatkan kinerja yang positif secara jangka panjang, mereka mendapatkan penilaian analis yang tinggi. Secara objektif, grafik ini menyajikan referensi saham berisiko rendah (*low-risk*) yang ideal bagi investor yang memprioritaskan kepastian pendapatan dan stabilitas modal di tengah dinamika pasar sepanjang tahun.

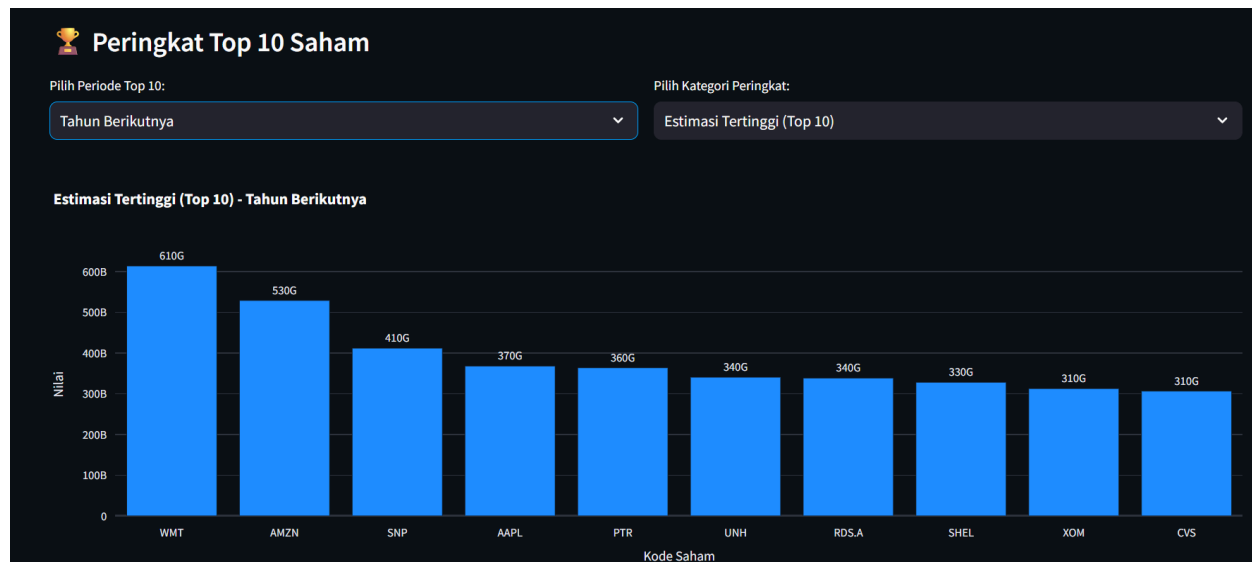


Gambar 35 Analisis 10 Saham Estimasi Tertinggi pada Periode Tahun Berjalan

6.8 Analisis 10 Saham Estimasi Tertinggi pada Periode Tahun Berikutnya

Visualisasi ini menampilkan daftar 10 perusahaan dengan kapitalisasi pasar raksasa dunia yang diproyeksikan memiliki angka estimasi tertinggi oleh para analis untuk tahun depan. Melihat data prospektif tahunan tersebut, WMT tetap memimpin dengan proyeksi nilai estimasi yang masif, mencapai 610 Miliar. Posisi kedua ditempati oleh AMZN yang juga menunjukkan lonjakan di kisaran 530 Miliar, disusul oleh SNP di angka 410 Miliar. Daftar ini turut diperkuat oleh perusahaan berfundamental solid lainnya, seperti AAPL, PTR, dan UNH. Karena

emiten-emiten ini diperkirakan konsisten mencatatkan kinerja positif secara jangka panjang, mereka mendapatkan penilaian analitis prospektif yang tinggi. Secara objektif, grafik ini menyajikan referensi saham berisiko rendah (*low-risk*) yang ideal bagi investor yang memprioritaskan kepastian pendapatan dan pertumbuhan stabilitas modal untuk rencana investasi tahun depan.



Gambar 36 Saham Estimasi Tertinggi pada Periode Tahun Berikutnya

6.9 Saham dengan Analisis Terbanyak Kuartal Berjalan

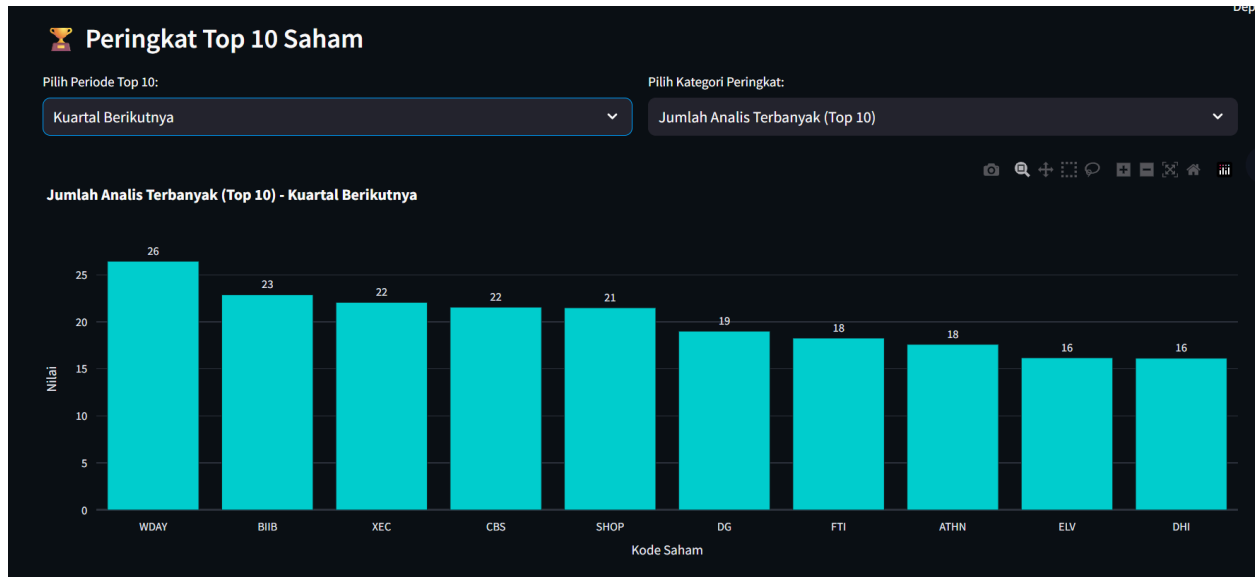
Visualisasi ini menampilkan daftar 10 perusahaan yang paling banyak dipantau dan dievaluasi oleh para analis pasar modal pada kuartal ini. Berdasarkan data pantauan tersebut, BIIB memimpin di urutan pertama dengan liputan sebanyak 27 analisis, disusul sangat ketat oleh WDAY yang dipantau oleh 26 analisis, dan CBS dengan 24 analisis. Daftar ini juga diisi oleh emiten terkemuka lainnya seperti XEC dan SHOP yang masing-masing dipantau oleh 22 analisis. Tingginya jumlah analisis yang memantau perusahaan-perusahaan ini mengindikasikan tingginya minat institusional, likuiditas pasar yang baik, serta tingkat transparansi informasi yang memadai. Secara objektif, grafik ini menyajikan referensi daftar saham yang sangat populer di kalangan profesional, sehingga cocok bagi investor yang mencari saham dengan aliran informasi dan evaluasi publik yang komprehensif.



Gambar 37 Saham dengan Analisis Terbanyak Kuartal Berjalan

6.10 Analisis 10 Saham dengan Analisis Terbanyak Kuartal Berikutnya

Visualisasi ini menampilkan daftar 10 perusahaan yang diproyeksikan akan paling banyak dipantau dan dievaluasi oleh para analis pasar modal pada kuartal mendatang. Berdasarkan data prospektif tersebut, terjadi pergeseran posisi di mana WDAY kini memimpin di urutan pertama dengan liputan sebanyak 26 analisis, menggeser BIIB ke posisi kedua yang dipantau oleh 23 analisis. Di posisi selanjutnya, XEC dan CBS bersaing dengan jumlah pantauan yang sama, yakni 22 analisis. Tingginya jumlah analisis yang memproyeksikan liputan pada perusahaan-perusahaan ini mengindikasikan kuatnya antisipasi pasar, minat institusional yang berkelanjutan, serta tingkat transparansi informasi yang memadai. Secara objektif, grafik ini menyajikan referensi daftar saham yang diprediksi akan sangat aktif dievaluasi oleh kalangan profesional pada kuartal berikutnya.



Gambar 38 Analisis 10 Saham dengan Analisis Terbanyak Kuartal Berikutnya

6.11 Analisis 10 Saham dengan Analisis Terbanyak Tahun Berjalan

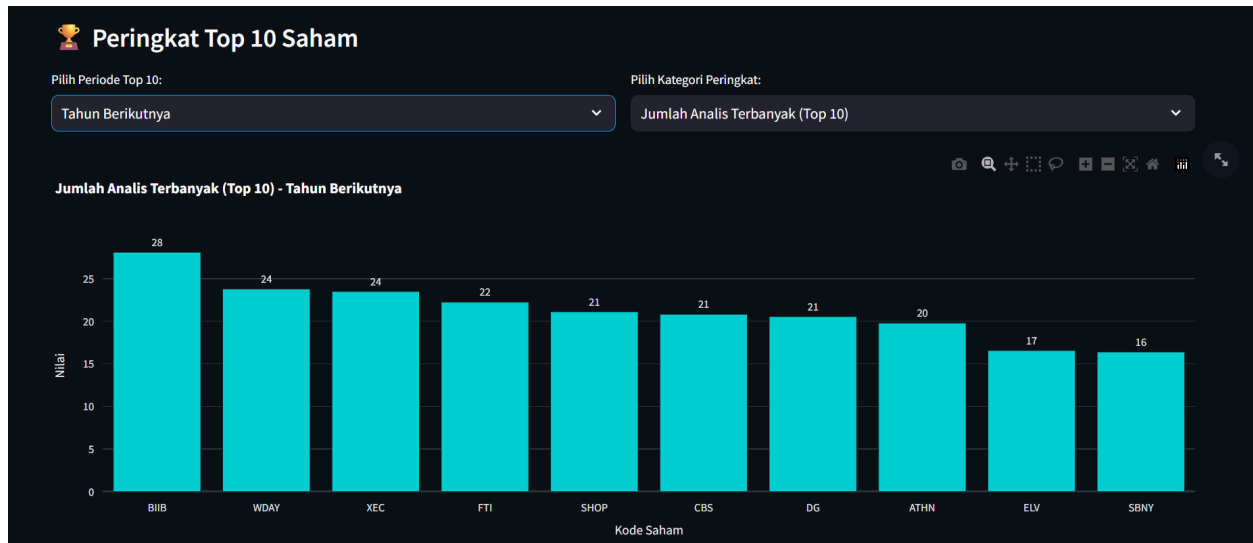
Visualisasi ini menampilkan daftar 10 perusahaan yang paling banyak dipantau dan dievaluasi oleh para analis pasar modal sepanjang tahun ini. Berdasarkan data tahunan tersebut, BIIB memimpin di urutan pertama dengan liputan sebanyak 26 analisis. Posisi ini disusul oleh WDAY yang dipantau oleh 24 analis, dan XEC dengan 23 analis. Di posisi tengah, terdapat persaingan ketat dengan jumlah pantauan yang sama, seperti pada CBS dan SHOP (keduanya 21 analisis), serta FTI dan ATHN (keduanya 19 analisis). Daftar ini juga diwarnai dengan masuknya emiten SBNY yang dipantau oleh 17 analis. Tingginya jumlah analis yang memantau perusahaan-perusahaan ini sepanjang tahun mengindikasikan kuatnya minat institusional yang konsisten, likuiditas pasar yang baik, serta tingkat transparansi informasi yang memadai. Secara objektif, grafik ini menyajikan referensi daftar saham yang menjadi fokus pengawasan utama di kalangan profesional selama tahun berjalan.



Gambar 39 Analisis 10 Saham dengan Analisis Terbanyak Tahun Berjalan

6.12 Analisis 10 Saham dengan Analisis Terbanyak Tahun Berikutnya

Visualisasi ini menampilkan daftar 10 perusahaan yang diproyeksikan akan menerima pantauan dan evaluasi paling intensif dari para analis pasar modal pada tahun depan. Berdasarkan data prospektif tersebut, BIIB memimpin di urutan pertama dengan proyeksi liputan mencapai 28 analisis. Di posisi berikutnya, WDAY dan XEC bersaing ketat dengan jumlah pantauan yang sama, yakni 24 analisis. Dinamika yang menarik juga terlihat di papan tengah, di mana FTI (22 analisis) menyalip ke posisi atas, disusul oleh tiga emiten sekaligus—SHOP, CBS, dan DG—yang masing-masing diproyeksikan akan dipantau oleh 21 analisis. Tingginya proyeksi jumlah analisis ini mengindikasikan ekspektasi pasar yang kuat terhadap emiten-emiten tersebut, menjanjikan likuiditas yang baik, serta pengawasan publik yang berkelanjutan. Secara objektif, grafik ini menyajikan referensi saham-saham yang diprediksi akan menjadi fokus utama kalangan profesional di masa mendatang.

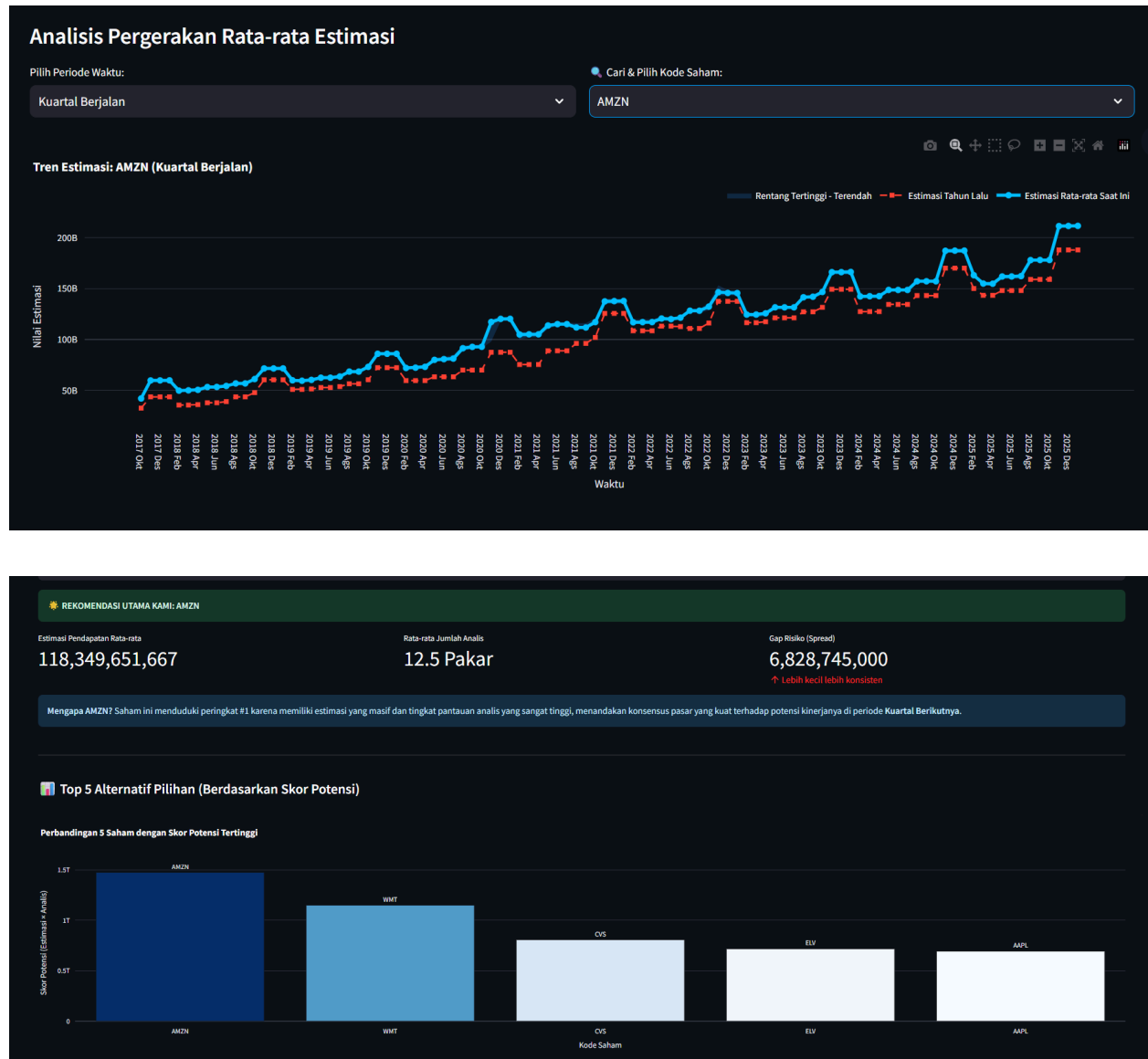


Gambar 40 Saham dengan Analisis Terbanyak Tahun Berikutnya

6.13 Pembahasan Dashboard Streamlit

Berdasarkan kumpulan visualisasi data yang berfokus pada proyeksi pendapatan, analisis menunjukkan adanya ekspektasi pertumbuhan fundamental yang sangat kuat, khususnya pada emiten raksasa berkapitalisasi besar. WMT secara absolut memimpin nilai estimasi baik dalam jangka pendek maupun panjang, dengan proyeksi tahunan yang tumbuh stabil dari 590 Miliar menjadi 610 Miliar untuk tahun depan. Namun, AMZN memperlihatkan agresivitas pertumbuhan yang lebih superior melalui lonjakan estimasi dari 460 Miliar menjadi 530 Miliar di periode yang sama. Menariknya, dominasi nilai fundamental ini tidak berbanding lurus dengan kuantitas pantauan pakar, di mana peringkat jumlah analisis terbanyak justru didominasi oleh saham saham seperti BIIB dan WDAY yang mengindikasikan tingginya aktivitas serta spekulasi institusional pada emiten tersebut. Mengacu pada visualisasi kedua terakhir yang secara khusus membahas Skor Potensi, kita bisa melihat lebih dalam bagaimana metrik ini bekerja dengan menggabungkan besaran estimasi pendapatan dan jumlah pantauan analis. Hasil kalkulasi dari grafik perbandingan tersebut menunjukkan bahwa AMZN secara meyakinkan menduduki peringkat puncak sebagai rekomendasi utama. Capaian Skor Potensi ini membuat AMZN berhasil menggeser WMT ke posisi kedua, membuktikan bahwa keseimbangan antara pendapatan yang masif dan tingginya kepercayaan konsensus analis jauh lebih berbobot daripada sekadar nilai nominal uang yang besar. Selain itu, visualisasi tersebut juga menyoroti CVS, ELV, dan AAPL yang berhasil mengamankan tempat di deretan lima besar alternatif investasi dengan

rasio yang menjanjikan. Keyakinan pasar terhadap posisi puncak AMZN ini turut divalidasi oleh indikator spread risiko yang sangat sempit di angka 6,8 Miliar, yang menandakan pandangan pakar yang sepakat dan minimnya kemungkinan kejutan fluktuasi harga. Ditambah lagi dengan bukti historis sejak 2017 yang konsisten mencatatkan tren revisi ke atas, seluruh data ini menyimpulkan secara tegas bahwa AMZN menawarkan portofolio paling ideal bagi instrumen investasi yang bertumbuh pesat sekaligus terukur keamanannya.



Gambar 41 Pembahasan Dashboard Streamlit

7. Kesimpulan

Dalam proyek ini, sistem Big Data untuk analisis data estimasi pendapatan dan penjualan saham berhasil dikembangkan dari tahap awal hingga akhir. Dataset awal berjumlah sekitar 6,6 juta record berhasil diproses menggunakan Apache Spark hingga menghasilkan data bersih sebanyak sekitar 5,7 juta record yang siap digunakan untuk analisis. Hasil pemrosesan kemudian disimpan secara terintegrasi ke dalam database SQL menggunakan MySQL/TiDB dan database NoSQL menggunakan MongoDB, sehingga alur penyimpanan data menjadi lebih fleksibel.

Berdasarkan hasil analisis, sistem mampu menghasilkan beberapa insight, seperti saham dengan estimasi rata-rata tertinggi, saham dengan estimasi paling stabil, serta saham dengan pertumbuhan estimasi tertinggi. Pada visualisasi dashboard Streamlit, hasil analisis kuartalan ditampilkan agar lebih mudah dipahami, termasuk peringkat saham berdasarkan estimasi dan jumlah analisis. Secara keseluruhan, sistem ini menunjukkan bahwa teknologi Big Data dapat membantu mengolah data saham dalam jumlah besar menjadi informasi ringkas dan terstruktur sebagai alat bantu analisis awal.